

REAL-TIME VIDEO ANALYTICS AT SCALE: DESIGN, DEPLOYMENT AND EVALUATION

by

Seyed Mohammad Pakdaman

A thesis submitted in conformity with the requirements
for the degree of Master of Applied Science
Graduate Department of Edward S. Rogers Sr. Department of Electrical and Computer
Engineering
University of Toronto

© Copyright 2020 by Seyed Mohammad Pakdaman

Abstract

Real-time Video Analytics at Scale: Design, Deployment and Evaluation

Seyed Mohammad Pakdaman

Master of Applied Science

Graduate Department of Edward S. Rogers Sr. Department of Electrical and Computer

Engineering

University of Toronto

2020

A key determinant of a video analytics application success lies in its ability to operate in a real-time fashion and must do so, regardless of the number of video streams the system needs to process. Hence, there is a need for a scalable and distributed video analytics system that can leverage the power of the cloud and thus successfully deliver on its promise. This work addresses this challenge by taking a bottom-up approach in defining the required system architecture and design for enabling large-scale execution of video analytics jobs in the cloud in a distributed and scalable fashion. The proposed system is designed by leveraging the existing open-source big data technologies and is deployed and evaluated to shed light on several intrinsic aspects of modern real-time video analytics applications.

Acknowledgements

To my loving family for believing in me and wholeheartedly supporting me in any endeavour I undertake!

I would like to thank my thesis supervisor, Dr. Alberto Leon-García, for providing me with this opportunity as well as my committee members Dr. Paul Chow and Dr. Elvino Sousa. I would likewise like to thank the supervisory team, Dr. Saeedeh Parsaeefard, Dr. Ali Tizghadam and Dr. Hadi Bannazadeh. Finally, I want to thank all my colleagues in the Network Architecture Lab at the University of Toronto.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Research Objective	1
1.3	Proposed Research	2
1.4	Organization	3
2	Related Works	4
2.1	Overview	4
2.2	Review and Discussion	4
2.3	Summary	7
3	System Requirement Analysis and Proposed Architecture	9
3.1	Overview	9
3.2	System Objectives	9
3.3	System Business/Functional Requirements	10
3.4	System Non-functional Requirements	11
3.5	System Architecture Design	12
3.5.1	Service Orchestration and Management	12
3.5.2	Data Streaming and Message Queues	14
3.5.3	Monitoring and Alerting	15
3.5.4	Data Pre-process and Enrichment	15
3.5.5	Data Storage	16
3.5.6	Video Analytics	16

3.5.7	Data Security	17
3.5.8	Presentation	18
3.6	Summary	18
4	Proposed System Design	20
4.1	Overview	20
4.2	System Logical Design	20
4.2.1	Proposed System's Data Flow	20
4.2.2	Logical Design of Service Orchestration and Management	21
4.2.3	Logical Design of Data Streaming and Message Queues	22
4.2.4	Logical Design of Monitoring and Alerting System	23
4.2.5	Logical Design of Data Storage	24
4.2.6	Logical Design of Video Analytics	29
4.2.7	Logical Design of Data Pre-processing and Enrichment	30
4.2.8	Logical Design of Data Security	30
4.2.9	Logical Design of Presentation	31
4.3	System Physical Design	31
4.3.1	Physical Design of Service Orchestration and Management	31
4.3.2	Physical Design of Data Streaming and Message Queues	33
4.3.3	Physical Design of Monitoring and Alerting System	38
4.3.4	Physical Design of Data Storage	43
4.3.5	Physical Design of Video Analytics	45
4.3.6	Physical Design of Data Pre-Processing and Enrichment	50
4.3.7	Physical Design of Data Security	51
4.3.8	Physical Design of Presentation	53
4.4	Summary	53
5	System Evaluation and Results	55
5.1	Overview	55
5.2	Physical Infrastructure and Testbed	55
5.3	Evaluation Methodology	58

5.4	Experimental Design	62
5.4.1	System’s Input Source	62
5.4.2	Workloads	63
5.5	Evaluation Results and Discussion	65
5.5.1	Video Analytics Component	65
5.5.2	Network Bandwidth Utilizations and Incurred Delays	73
5.5.3	Leveraging the Video Analytics Results	78
5.5.4	Real-time Presentation and Visualization	82
5.5.5	Implications of Parallelism on the Real-time Aspects of the System	83
5.5.6	Different Cloud Deployment Settings	85
5.6	Summary	87
6	Conclusions and Future Work	90
6.1	Conclusions	90
6.2	Future Work	92

List of Tables

5.1	System Configurations for the SAVI Acquired VM Instances	57
5.2	System Hardware and Software Configuration of SciNet’s Physical Machine . . .	57
5.3	System Deployment’s Dedicated Graphics Processing Unit (GPU) Specifications	58
5.4	Summarized Description of the Workloads Used in the System’s Evaluation . . .	64
5.5	Time Measurements for the End-to-end Delay’s Constituting Components	77
5.6	Measured Values for a Different Number of Active Parallel Video Analytics Workers	84

List of Figures

3.1	High-level Overview of the Real-time Video Analytics System	10
3.2	Block Diagram of the Proposed Real-time Video Analytics System	13
4.1	Proposed System's Abstract Data Flow Diagram	21
4.2	Proposed System's Logical Design of Data Storage Component	25
4.3	Proposed System's Monitoring and Alerting Toolchain	38
4.4	Proposed System's Physical Design of Data Storage	43
4.5	Video Analytics Component Block Diagram	46
5.1	Proposed System Deployment's Physical Infrastructure and Testbed	58
5.2	Constituting Components of the End-to-end Delay	60
5.3	Worker Threads in the Video Analytics Component	65
5.4	Inference Time for a Varying Number of Active Video Analytics Worker Threads	67
5.5	Video Analytics (VA) Component Data Flow Diagram	69
5.6	Deployed System's Average Processing Time Required for each Video Frame . . .	70
5.7	Deployed System's Throughput Measurements	72
5.8	Box-and-whiskers for the Deployed System's GPU Resource Utilizations	74
5.9	Secure vs. Insecure Communication's Network Bandwidth Utilization	75
5.10	System's End-to-end Latency for Messages Sent Through the Kafka Broker and Incurred Network Bandwidth Utilizations	76
5.11	System's Detected and Tracked Car Objects in the Highway	80
5.12	Deployed System's Dashboards used for Real-time Data Presentation	83

Chapter 1

Introduction

1.1 Motivation

The introduction of increasingly more powerful GPUs and the emergence of more capable computer vision algorithms based on deep neural networks have resulted in a substantial increase of video analytics applications in various use cases and domains [1–16]. Video analytics applications are inherently time-sensitive and, thus, need to perform in a real-time fashion. Consequently, a large body of work has been dedicated to enabling these algorithms’ real-time operations by various means. That includes proposing more efficient vision algorithms, devising systems to adaptively and intelligently perform the algorithms’ parameter tuning, leveraging cloud computing, and introducing fast and efficient cloud system architectures and designs [17–30]. Nevertheless, notwithstanding the several recent advancements and the abundance of open source big data frameworks and technologies today, meeting the real-time requirements of video analytics applications in the cloud remains a challenge.

1.2 Research Objective

Design, deploy and evaluate a scalable and real-time video analytics system leveraging the existing open-source big data technologies in the cloud with the capability to support any number of required video analytics workloads.

1.3 Proposed Research

This work aims to enable real-time video analytics by taking a bottom-up approach in designing a scalable and distributed system leveraging the existing open-source big data technologies.

The system's proposal is accomplished by taking a systematic approach towards the system's design, with due consideration of all inherent requirements and limitations of the modern video analytics applications.

The designed system is deployed to enable a thorough system evaluation, thereby shedding light on several *practical aspects* of real-time and distributed video analytics systems.

The contributions resulting from this work are expected to aid with answering research questions, including and notwithstanding the below-outlined questions:

- *What system architecture components are required to satisfy the functional and non-functional requirements of a real-time video analytics application? For the system architecture, how must the system's logical and physical design be carried out to satisfy system functional requirements?*
- *More specifically, how is the system enabled to deliver on its real-time requirements? What protocols, methodologies or algorithms facilitate that?*
- *Which existing big data technologies and in what ways may enable real-time operation of a video analytics application?*
- *What are the main contributing components to the end-to-end delay experienced by a real-time video analytics application in practice?*
- *What is the performance exhibited by each of the system's constituting component in the critical path? What are the likely performance bottlenecks, and how should they be mitigated?*
- *What algorithms and in what ways can be used to derive value and extract knowledge from the streams of video received by such a system?*
- *How may the inferred knowledge from vision algorithms be used to build more complex and intelligent applications with substantial potential to improve our daily lives?*

1.4 Organization

This document is organized as follows. The set of related works in the literature are presented and discussed in Chapter 2. In Chapter 3, the proposed real-time video analytics system's requirements are discussed and therefore used to devise the required system's architecture. Consequently, system's logical and physical designs are carried out in Chapter 4. The proposed system is therefore implemented, deployed and evaluated in Chapter 5. Ultimately, the document herein concludes in Chapter 6 by presenting the conclusions of this work and outlining the future directions.

Chapter 2

Related Works

2.1 Overview

In this chapter, the relevant literature pertinent to this work is presented and discussed. First, the emerging video analytics applications and their use cases are described. Therefore, the existing frameworks and methodologies used to address the real-time video analytics systems' inherent challenges are presented and discussed.

2.2 Review and Discussion

Video analytics have enabled several useful applications in a wide variety of domains. For instance, video analytics have been used to monitor farms and livestock [1–3]. For instance, Farmbeats [2] is proposed as a framework to leverage video analytics for smart and data-driven agriculture. In addition to cameras, Farmbeats uses drones to survey the farm area and transmit it to a central processing system. Therefore, the results obtained from video sources are aggregated with data extracted from a large set of deployed sensors in the farm, to yield valuable insights into the farm, hence, boosting agricultural productivity.

A large body of work has focused on enabling smarter and automated vehicle traffic management in cities and crowded areas [4–10]. Arinaldi et al. [5] use computer vision techniques to automate the extraction of information from cameras monitoring vehicles' traffic, thereby aiding policymakers with summarized statistics on vehicle counts, vehicle classes, and vehicle speed

estimations. They use two different video analytics approaches and discuss and compare the accuracy of the results obtained with each approach’s employment.

Similarly, Wan et al. [6] build a real-time highway surveillance system to extract traffic statistics, such as vehicle counts and estimating vehicle speeds. The system is intended to work autonomously and used for generating traffic alerts when, for instance, a monitored vehicle has stopped. Their work additionally proposes an algorithm to perform automatic camera calibration, thereby receiving a crisp and sharper image from the camera as the characteristics of the monitored region change.

Likewise, Sevillano et al. [7] build a system to enable smarter traffic management by spotting on-street vacant parking spots in a video feed. The work aims at improving the detection accuracy that is achieved by incorporating and fusing data from additional sensors deployed on the street.

Other works focus on using video analytics for crowd monitoring and public safety [11–16]. Cheong et al. [11] implement a system to automate person identification, tracking and counting in crowded areas based on live video streams. Zhang et al. [12] report and review a more extensive set of video analytics applications for public safety in recent years.

Musalem et al. [13] instead leverage video analytics on in-store video recordings to monitor customer-employee interactions to measure and assess employee’s productivity and thus, infer how the level of assistance a customer receives impacts the overall revenue of a business.

However, an inherent challenge in each of these applications is carrying out the required compute in a scalable and real-time fashion, given the significantly demanding nature of computation required for video analytics jobs. As such, a large body of work in the literature focuses on evaluating or proposing frameworks and methodologies to facilitate and enable large-scale video analytics to be performed in real-time [4, 17–30].

Wang et al. [19] instead propose a system architecture to deal with inherent privacy concerns in video analytics applications, thus, allowing for creating privacy-aware video analytics. In essence, the work aims to automatically identify potential leaks in captured videos from a privacy standpoint and, hence, eliminate any sensitive information therein. The above anonymization process is referred to as denaturing a frame and involves blurring or obfuscating parts of an image, such as the faces of people present in an image. In their system, a frame’s denaturing

takes place prior to it being forwarded to a presumably untrustful and external video analytics software.

Anjum et al. [26] introduce a cloud video analytics framework to automate the object detection and classification process in a video to abstract away the complexities and thus make it easier for end-users to perform these jobs. The framework is aimed at leveraging the cloud to maximize achievable throughput. However, a drawback of this work is the relatively low support offered for the real-time video streams. Therewith, the video frames are first stored in a cloud object storage and, therefore, retrieved for analysis. Likewise, in this case, the user is not presented with real-time analytics results; instead, it is simply notified upon successful termination of the work. The work also evaluates GPUs' use instead of CPUs for processing and discovers that the former outperforms the latter.

Tornado [30] is proposed as a scalable and flexible framework for large-scale video analytics in the cloud. Tornado is designed as being service-oriented and hence, easily portable and deployable to any cloud provider. A drawback of the Tornado framework is a potentially over-inclusive approach in the use of big data technologies, eventually resulting in an exceedingly complicated design that will be likely hard to operate and maintain. Moreover, the use of HDFS as the underlying storage system for video frames may not be optimal or efficient, given that on the one hand, HDFS stores chunks of data as blocks, typically in the range of 64 to 256 megabytes. On the other hand, video streams are customarily composed of 30 to 60 frames per second, with sizes on the disk of at most few megabytes at most. Therefore, the storage and retrieval of frames required for analysis for HDFS are likely to incur large overheads both in terms of time and network bandwidth.

LAVEA [31] framework instead deals with the latency aspect of video analytics job and is thus proposed as a “latency-aware” platform. LAVEA is designed on the premise that the considerably strict timing constraints of video analytics applications would inevitably require edge-computing to reduce latencies. To that end, LAVEA internally uses a variety of optimization techniques to find an optimal setting for offloading of video analytics tasks among edge nodes such that latency can be minimized.

VideoStorm [23], on the other hand, is proposed as a framework to intelligently and efficiently schedule video analytics jobs such that the specific requirements of each job can be met.

VideoStorm does so based on the specific configuration parameters of the underlying vision algorithms, or *knobs*. A knob can be anything from image resolution, frame rate to specific tuning parameters of the vision algorithm. The entire space of the configuration parameter is defined as a *query configuration* in VideoStorm. Considering that different jobs may have varying requirements in terms of quality or maximum tolerated lag, VideoStorm does hence, leverage that make intelligent decisions on the distribution of jobs and allocation of required resources.

Likewise, Chameleon [22] is proposed as another framework to intelligently optimize resource allocation with minimal impacts on the accuracy of video analytics jobs. Chameleon is video-adaptive, and, with that, it performs dynamic changes to the configuration of knobs or allocation of resources. Chameleon evaluation demonstrates it as capable of achieving 20-50% more accuracy by intelligently increasing the required resources or otherwise achieving the same level of accuracy by, however, 30-50% of resources.

The works discussed above and others in the literature address one of the several challenges inherent to large-scale and distributed video analytics applications. This work complements the frameworks discussed above by taking a bottom-up approach in designing a real-time video analytics application that is scalable and distributed. Therefore, it offers an alternative approach to scalable video analytics by leveraging the existing open source big data technologies and thus, shedding light on the different functional aspects of typical real-time video analytics applications through the deployment and evaluation of the proposed system.

2.3 Summary

The chapter discussed and presented the pertinent body of work in the literature. The existing literature showed a growing number of video analytics applications across several use cases and domains. The preceding included using video analytics to boost agriculture, enable data-driven management of vehicle traffic in the cities, promote the driver's experience by intelligently locating vacant parking spots on the street or improve the customer's experience by leveraging video analytics and thus deriving knowledge and insights into the quality of customer-employee interactions of a business, thereby, heightening the customer's experience.

Consequently, given the computationally-intensive nature of the required video analytics workloads, several discussed works in the literature instead focused on optimizing and enabling the efficient carrying-out of the required computation. Moreover, given the usually sensitive nature of the data collected in video analytics applications, several works were geared more towards proposing methodologies and strategies that enable and facilitate compliance with the privacy requirements.

The above efforts ranged from framework proposals optimizing the use of the required computing resources by performing self-adaptive and dynamic tuning of the video analytics workload's configuration on the fly, perform intelligent scheduling of several video analytics jobs to satisfy the requirements of each while also optimizing the utilization of the underlying hardware resources or alternatively, devising frameworks to perform the intelligent distribution of tasks between the cloud and the edge, thereby, reducing minimizing the delays experienced by the application.

Therefore, the proposed system in this work was described as a pragmatic effort to complement the above endeavours through the design of a scalable and real-time video analytics system that leverages the existing open-source big data technologies in the cloud and is designed as being capable of supporting any number of required video analytics workloads. Moreover, the thesis work herein focuses on delving deeper into practical performance requirements and characteristics of large-scale video analytics applications in the cloud through the actual implementation, deployment and evaluation of the proposed system in the cloud.

The subsequent chapter analyzes and defines the detailed set of system's requirements, which is therefore used to define the proposed system's architectural requirements.

Chapter 3

System Requirement Analysis and Proposed Architecture

3.1 Overview

The architectural design of the proposed real-time video analytics system is the main focal point of this chapter. The architectural design of the system is carried out in stages. As a first step, the system objectives are defined in Section 3.2. In Section 3.3 and 3.4, the user/business requirements, the functional and the non-functional requirements of the proposed system are outlined and discussed. The outcome of the latter is therefore used in Section 3.5 to devise the system's architecture. Specifically, the system's architecture design is carried out posterior and consequent to the definition of the foregoing requirements.

3.2 System Objectives

Figure 3.1 illustrates a high-level overview of the proposed real-time and scalable video analytics system in this work. As shown in the figure, the system is designed to carry out the set of video analytics jobs requested by the system user. The video analytics is performed by the system in a distributed and scalable manner and on any number of supplied video streams.

The user-provided information is used by the system to authenticate with and retrieve the intended set of video streams. As a practical matter, the above video streams comprise

the footage from surveillance cameras that reside on the user/customer premises continuously monitoring a region of interest. The video analytics provide the system user with meaningful insights into varying set of events as they occur in each of the cameras' monitored regions.

The proposed system is designed to scale on-demand and perform the required set of video analytics jobs on input video streams, hence, returning the obtained analytics results to the system's user in real-time.

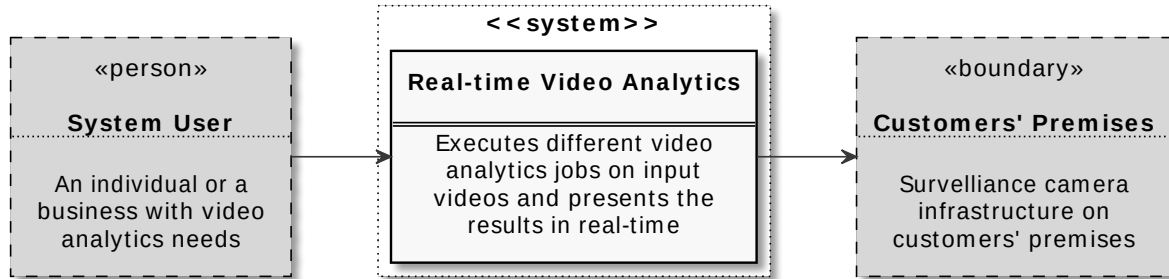


Figure 3.1: High-level Overview of the Real-time Video Analytics System

3.3 System Business/Functional Requirements

The section herewith briefly discusses both the user and the functional requirements of the proposed system. In general terms, the user/business requirements serve to determine the system's functional requirements. The user requirements are domain and application specific. Therefore, for purposes of this work, the user requirements are presumed as closely following the defined system objectives as set out in Section 3.2.

The Functional Requirements (FRs) are derived from the user requirements as noted above and in practical terms, define the functionalities a system must accomplish. For the purposes of this work, a set of broadly used and defined requirements in practice are assumed to comprise the system's functional requirements.

The most primitive objective of a video analytics application is to intelligently infer and obtain knowledge from an input video source. The acquired knowledge often consists of information on objects that are inferred as being present in a video stream. Moreover, more detailed statistics about objects detected in a video may occasionally be desired.

For instance, it may be useful to know for a predefined set of object classes, which particular

class does a detected object belong to. Likewise, information on the detected objects' relative location to the camera may be sought or desired. Moreover, a video analytics application may require that movements of a particular detected object be systematically tracked, as the object moves throughout the camera's monitored region.

Therefore, this work assumes all of the above mentioned requirements as making part of the proposed system's functional requirements.

3.4 System Non-functional Requirements

The Non-functional Requirements (NFRs) of the proposed system; in essence, define the set of qualities and attributes the proposed system is anticipated to possess. Intuitively, the proposed system will have a large set of non-functional requirements associated with it. However, for conciseness and brevity, the discussion of non-functional requirements herein, highlights the imperative while stripping out the non-essential.

The system's real-time nature inevitably demands that the system's components exhibit *high throughput* and *low response times*. The degree to which the system may exhibit each of the former attributes is coarsely determined by the particular design choices made in each component's design. The latter, on the other hand, is often predicated upon the aptness of the system's configuration and the optimal tuning of the system components throughout the system's deployment stage.

Besides, the proposed system is expected to enable the seamless integration of any existing video source used for analysis. Consequently, the system must *scale horizontally* in response to any eventual surges in the systems' input load. Similarly, the system must be *elastic* and as such, quickly adapt to changes in the input load, *i.e.*, ensure the available computing resources indeed match the system's demand and thus limit an eventually wasteful utilization of the computing resources whenever the system is operating under relatively moderate loads and vice-versa.

Fault-tolerance, *Fault Management*, *Resilience* and *Robustness* comprise the inherently essential attributes of a system, often defining how well the system can deliver on its promise. As a matter of practice, computing systems do frequently undergo failures and do commonly experience interruptions for a large variety of reasons. Whereas the system's robustness can

gradually be improved over time, the fault manageability and resilient aspects of a system are required for cutting down on the often lengthy times associated with the manual inspection, diagnosis and rectification of software issues.

The extent to which a system may exhibit each of the attributes mentioned above, does ultimately boil down to the aptness and appropriateness of choices made in the system's design. Consequently, both the system's logical and physical design decisions are made with due consideration of all the critical system's characteristics and qualities discussed above.

3.5 System Architecture Design

The architectural design of the proposed real-time video analytics system is carried out in this section, after due consideration of the above-stated set of requirements. Figure 3.2 illustrates at a high-level, the set of the required architectural components as identified in the preceding sections. The detailed description of each system architecture component is provided in sections that follow.

3.5.1 Service Orchestration and Management

The service orchestration and management component is a fundamental system component, principally responsible for effectively leveraging the underlying physical infrastructure to spawn all of the system's required services throughout the entire set of provisioned system's computing resources.

The component serves as the single point of management and administration of services and accommodates the distribution and scaling of each of the system's constituting services. Moreover, the component should carry out the above requirements in a manner that facilitates the efficient utilization of the computing resources, therefore, avoiding the wasteful usage of expensive computing resources.

As such, the proposed system makes use of containers for the implementation and deployment of the system's comprising components. Containers leverage the Linux kernel's built-in containment feature, often referred to as Linux Containers or LXC, for short, to provide operating-system-level virtualization. The technology allows us to create and execute multiple

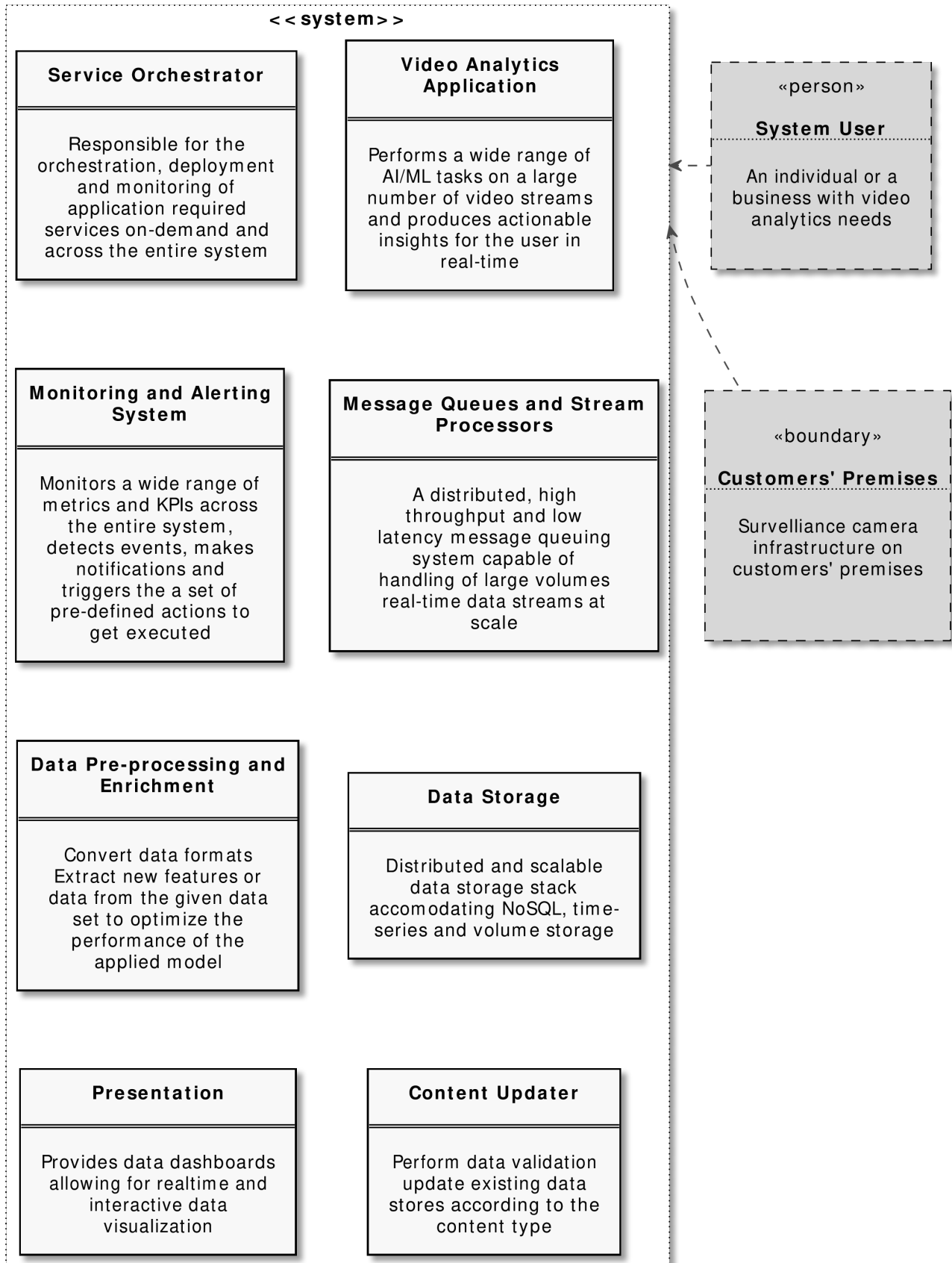


Figure 3.2: Block Diagram of the Proposed Real-time Video Analytics System

isolated Linux Virtual Environments (VE) on a single computing host.

The latter, however, is different from the full system virtualization that is provided by a hypervisor. The hypervisor allows the creation of multiple Virtual Machines (VMs), where each can have concurrent and shared access to a single physical machine's hardware resources. Each VM can be configured with the operating system of choice and can have a particular set of software stacks installed and configured.

Virtualization provides for nearly perfect isolation and decoupling of services created as virtual machines from the underlying hardware and specifics of the deployment scenario where aforesaid services will execute. However, VM's overheads resulting from the inevitable need for a full operating system stack is often not wanted or desired. Consequently, the incurred overheads mentioned above can outweigh the benefits achieved through virtualization, notably, wherever the same operating system in a particular deployment scenario, is used by all the required services running as VMs.

In contrast to VMs, however, containers in a computing host all share the same operating system. The built-in resource isolation features of the Linux kernel, such as the control groups (**cgroups**) and the kernel namespaces, provide mechanisms to control and therefore restrict the set of computing resources that is accessible by a container, thereby creating perfect isolation of processes running as containers on the same host.

Therefore, a container can be regarded as a lightweight process that runs in isolation of the other co-existing processes. The above isolation applies, among other things, to the processes, the file system, the inter-process communication (IPC), and the network.

Finally, reliance on containers for the deployment of all system's required services has several advantages as noted above and allows the creation of unified pipelines through the consolidation of system components.

3.5.2 Data Streaming and Message Queues

It is paramount to ensure the system components can exchange data through highly available, distributed and scalable data pipelines. The Data Streaming and Message Queues component is devised so that it can enable high throughput data flows across all the system's comprised components. Moreover, the proposed video analytics system's real-time requirements require

that the end-to-end delay and the latency incurred through each data pipelines be minimal.

The component, therefore, implements a publish-subscribe messaging pattern. In such a pattern, the producers of the data or messages, called publishers, do not directly transmit the messages to a specific recipient. Instead, the content is published under a particular topic name to the message broker(s), whereto the intended recipient(s) of the content, typically referred to as the topic subscribers, do subscribe and download the published messages. The latter approach allows for the content published by a producer to be consumed by several subscribers without the need for a consumer's awareness of who the original publisher of the content is and vice-versa.

Whereas the employment of a publish-subscribe messaging pattern in the system facilitates the creation of flexible data pipelines, special care must be taken to warrant the resulting incurred overheads, do not negatively impact the system's real-time requisites.

3.5.3 Monitoring and Alerting

An indispensable prerequisite in guaranteeing the system's correct operation and functioning is implementing strategies to perform extensive and pervasive monitoring of system-wide components.

The Monitoring and Alerting component of the system must continuously monitor and collect the system's critical performance indicators and metrics. Besides, it should do so in a real-time fashion, without incurring unnecessary delays. The latter is pivotal to the operation of the alerting subsystem of the component. The alerting system will trigger alerts based on a pre-established set of rules defined a priori on the entire set of collected metrics.

As the name suggests, the alerts can merely be composed of notifications sent out to the system's administrator. However, alerts can also be more instructive and, instead, trigger automatic mitigatory actions on-the-fly, such as the restarting or re-deployment of any of the system's services that have either crashed or are undergoing issues on the runtime.

3.5.4 Data Pre-process and Enrichment

Each of the proposed systems' constituting components is anticipated to produce large volumes of data, and each chunk of data will likely have a distinct format and structure. Therefore,

the Data Pre-process and Enrichment component is devised as an architectural component to perform any required data cleaning, transformation and integration tasks on the system's generated data.

The Data Pre-process and Enrichment component is essential to bridge any existing communication gaps resulting from different data formats and protocols employed across the system components. Moreover, the component is responsible for taking care of data aggregation and enrichment tasks on the streaming data wherever required and applicable.

3.5.5 Data Storage

The Data Storage component is an integral and fundamental component of any large-scale and distributed analytics system. The sole component's goal is to provide scalable and fault-tolerant ways to persist and store data.

The particular type of data storage used is predicated upon the specific type of data being stored as well as the anticipated manner whereby the persisted data will be queried and retrieved in future. Therefore, the Data Storage component's development will most likely involve the integration and utilization of several distinct types of data stores.

3.5.6 Video Analytics

The Video Analytics component lays at the core of the proposed system and serves as the primary source of data analytics results and insights derived from the input video streams. Therefore, the component comprises a rich set of video analytics functions and employs a wide range of video analytics algorithms to derive meaningful and intelligent insights into the occurring events in the camera's monitored regions as they are manifested.

At a broad level, the video analytics application performs two primary video analytics functions. It first utilizes the state-of-the-art algorithms to detect and classify objects present in each of the retrieved video frames. Therefore, it makes use of robust and publicly available pre-trained models to accurately determine the particular class a detected object belongs to.

Second, it performs the visual tracking of objects detected by the formerly discussed component. It employs the state-of-the-art object tracking algorithms to allow the application

keep track of each of the detected objects in a frame and therefore, re-identify them in the subsequent frames as those objects move throughout the camera's viewing range.

Moreover, the video analytics component does provide for mechanisms to retrieve video from a wide range of video types used as input source including for instance, video streams produced by a surveillance camera device. The particular video source is tightly coupled with the particular application and use case of the proposed system and it varies from one deployment to the other. As such, it is incumbent that the system can support the standard and widely-adopted communication protocols used in practice in the retrieval and distribution of real-time video contents and streams.

Lastly, the video analytics jobs are considerably computationally-expensive jobs and therefore, the system's extensibility attributes demand that the output of each video analytics job such as detection or tracking be persisted for long-term use so that, past executed jobs may be efficiently reproduced without need for perform the actual processing again.

The latter is a particularly essential step needed for enabling the future works to extend or amend the existing underlying algorithms and procedures incorporated into the video analytics component of the proposed system.

3.5.7 Data Security

The proposed system's core objective is to perform various real-time video analytics jobs on any particular set of video stream feeds used as input. The videos in all likelihoods originate from video cameras used for the surveillance or monitoring of a particular region of interest. As such, the video streams most likely involve sensitive information that needs to be safeguarded against unauthorized access or protected from eventual cyber attacks.

The Data Security component is therefore, an integral system component primarily responsible for ensuring the safety and security of data throughout the entire system. The component achieves the before-mentioned goals by performing the appropriate encryption and authentication of all the traffic that passes through publicly accessible networks and, hence, restricts and limits non-trusted sources' access to the data generated by each system component.

3.5.8 Presentation

The presentation component serves as the system's interface to the end-user, wherein the end-user is presented with the insights and data analytics results obtained by executing all the required video analytics jobs. An essential consideration for the presentation layer's design stems from the understanding of the mission it must accomplish as well as the particular entity whereto is interfaced. As noted above, the presentation layer is intended to enable the end-user query, visualize and interactively analyze the obtained analytics results. As such, essential to the presentation component functioning is its support for a wide variety of visualization tools and graphs, whereby the end-user may interactively explore and manipulate the data in real-time.

3.6 Summary

In this chapter, the system architecture of the proposed system was introduced and discussed. A systematic approach was taken to determine the architectural requirements and the needed components for the proposed real-time and scalable video analytics system in the cloud.

The first step involved a concise definition of the system objectives. Therefore, the system requirements were defined and discussed. The system requirements comprised the business requirements, as well as the system's functional and non-functional requirements.

The system's business and functional requirements were defined, and assumptions were made along the way by necessity. The business and functional requirements were presumed as consisting of primitive yet widely adopted video analytics functions, such as object detection, classification and object tracking in the input video streams.

Therefore, the non-functional requirements of the system were discussed and defined. That included the proposed system's scalability, elasticity, fault-tolerability, resilience, availability and performance. The non-functional requirements did also additionally include extensibility and security.

The already defined and established system's requirements were consequently used to define and develop the proposed system's architecture. First, a set of essential system architectural components were introduced.

The proposed system's architecture comprised several architectural components, each devised

to effectively ascertain the proposed system's ability to deliver on the pre-established set of system requirements.

The proposed system's architecture included components for the orchestration and management of the required system services throughout the cluster of computing nodes; distributed high throughput and low latency data pipelines to enable the communication among the system's comprised services, and a video analytics component to enable the efficient and high throughput execution of the video analytics jobs.

The system architecture also included data security components to shield and safeguard the system-wide data from unauthorized access, scalable and distributed storage for long-term persistence of data, and presentation components to enable real-time and interactive exploration, analysis, and visualization of the system generated data.

Finally, the need for the incorporation of each architectural component in the design was thoroughly discussed and motivated throughout this chapter.

Chapter 4

Proposed System Design

4.1 Overview

In this chapter, the design of the system is carried out in two consecutive stages. The first stage involves the system’s logical design and is thoroughly discussed and accomplished in Section 4.2. Therefore, the physical design of the system is carried out in Section 4.3.

4.2 System Logical Design

4.2.1 Proposed System’s Data Flow

Figure 4.1 illustrates an abstract overview of the data flow among the system components. The diagram represents at a broad level how the defined components in Chapter 3 will communicate and interface with each other. As illustrated in Figure 4.1, first the video analytics component of the proposed system securely connects the security cameras residing in the customer’s premises and retrieves the real-time video footage captured therefrom.

Therefore, the system performs the required video analytics jobs on the input video frames. The analytics job results are then forwarded to the proposed systems’ Message Queues and Stream Processors component. In simple terms, the above component serves as the message broker that facilitates high-throughput and low-latency communication among the system components.

Hence, the Data pre-processing and Enrichment component retrieves the published data

streams in real-time and performs the required data transformations or aggregations on each of the data streams. The processed data is ultimately forwarded to the Content Update component, responsible for updating the underlying Data Storage with data it receives in real-time.

On the other hand, the Monitoring and Alerting system collects the events and performance metrics throughout the system and directly interfaced with the Data Storage component for long-term data persistence purposes.

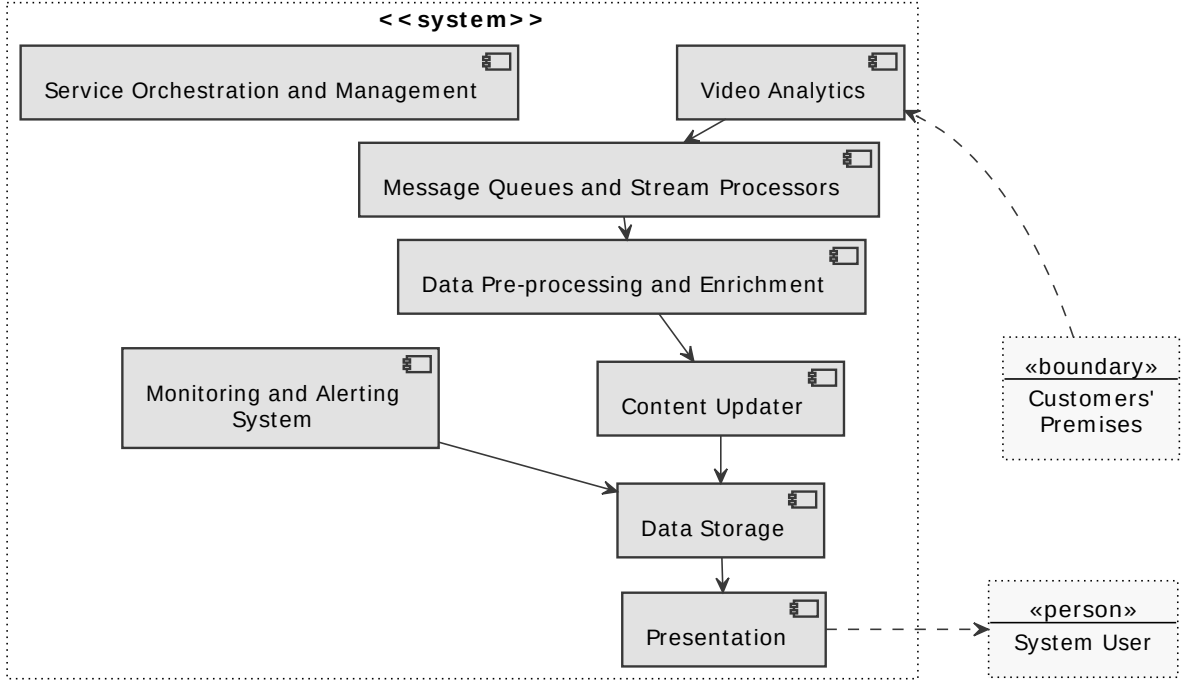


Figure 4.1: Proposed System's Abstract Data Flow Diagram

4.2.2 Logical Design of Service Orchestration and Management

The Service Orchestration and Management component is primarily responsible for the spawning of all the system-required components throughout the cluster of computing nodes. As discussed in Section 3.5.1, the component makes use of containers for the orchestration of services.

As far as the component's logical design is concerned, the component mentioned above implements tools to warrant the correct execution of the required services and re-spawning of the services in the event of failure. Moreover, the component provides a single centralized management system that features high-level and abstract APIs to manage application-wide services. The latter is particularly essential for the system's extensibility and maintainability,

especially when the comparatively large number of the system constituting components are taken into consideration.

The specific open-source technologies available for the management of containers are small and limited. Consequently, the component's logical and physical design are intertwined and hence for conciseness, further discussion of the component's design will be deferred to Section 4.3.1.

4.2.3 Logical Design of Data Streaming and Message Queues

The Data Streaming and Message Queues component of the proposed system enables creating unified data pipelines connecting all system components. The associated data pipelines facilitate the real-time and low latency flow of streaming data. Additionally, to satisfy the system's requirements, these data pipelines are tuned such that the data throughput can be maximized.

As discussed in Section 3.5.2, the component employs a publish-subscribe messaging pattern. Whereas several existing technologies employ a publish-subscribe messaging pattern, an essential consideration in enabling low latency data pipelines in the component's logical design is the particular communication protocol used in both the submission and the ingestion of data from the distributed message queues of the component. For instance, many of the existing message brokers technologies use the HTTP protocol and REST APIs to transmit and retrieve the data streams.

However, the overheads inherent to the HTTP protocol, which results from the required HTTP handshakes, can negatively impact both the required data pipelines' performance and scalability. Therefore, in the logical design of the Data Streaming and Message Queues component of the proposed system, the communication protocol stack employed by the particular message broker technology is an equally important consideration.

Other than efficiency measures such as the latency and throughput exhibited by the particular message broker employed, the degree to which the intended component is capable of providing delivery and ordering guarantees of messages needs to be taken into account.

The former guarantees that the message broker has indeed received the published message. On the other hand, the latter serves to guarantee the messages consumed from the distributed message broker are retrieved in the same order as they initially were published.

Each of the preceding attributes makes an essential characteristic the component must possess in order to satisfy the system's real-time and scalable functioning. Therefore, the preceding attributes lie at the core of the component's physical design carried out later in this chapter.

4.2.4 Logical Design of Monitoring and Alerting System

The performance metrics and statistics are collected in real-time at various system levels and can be consolidated into four broad categories as outlined below:

1. System-level Metrics
2. Container-Level Metrics
3. Video Analytics Runtime Metrics
4. Application-wide Analytics Results

The agglomeration of metrics collected from all the sources mentioned above results in a plentiful set of time-series system performance indicators. Therefore, the set is exploited and leveraged to perform continuous probing and monitoring of the system components, thereby enabling in-depth insights into eventual performance issues as they occur hence, aiding with the performance analysis and diagnosis.

A more detailed description of the particular type of metrics each of the above sets is comprised of is outlined below:

- **System-level Metrics and Logs:** System-level metrics contain detailed and granular statistics collected from various available computing resources, including the CPU, the memory, the disk and the network. The collected system-level metrics are essential and necessary for the continuous health monitoring of the individual nodes and the detection and diagnosis of the eventual performance issues thereof. Moreover, the set includes the complete set of logging information produced by varying subsystems of the running node and useful for both the debugging and the triggering of corresponding alerts whenever a module or component of the system encounters issues fulfilling its requirements.

- **Container-level Metrics and Logs:** The Container-level metrics, similar to the System-level metrics, contains statistics on the utilization of the multiple computing resources available on a node. However, the resource utilization is only measured for the processes running in a namespace and control group (*cgroup*) other than the original processes running on the host. As the reader can anticipate, the set of collected statistics results in a rich set containing important information on how each of the running containers on the node utilizes each of the accessible computing resources. Additionally, the running container's runtime logs are collected to enable both the performance diagnosis and reporting of the events wherever an exception or error in a containerized process impairs its correct functioning.
- **Video Analytics Runtime Metrics:** The Video Analytics Runtime Metrics includes all metrics collected at the runtime throughout the Video Analytics application's execution. It is mostly used for the performance diagnosis and health-checking of the application as it executes.
- **Application-wide Analytics Results** The set includes high-level results obtained through the execution of the analytics jobs on each video stream used as input to the application. For instance, the set includes both the quantity and the type of objects detected in a video stream at any particular point in time.

The collected metrics are used for the continuous health monitoring of all the system components. The accumulated statistics are then inputted to the alerting system, where several mitigatory actions may be taken based on a predefined set of rules.

4.2.5 Logical Design of Data Storage

The proposed system makes use of several types of data storage for the persistence and the swift retrieval of data to serve both the interactive and ad hoc queries.

Throughout the application's execution, an abundant set of data is generated by each of the proposed system's constituting components. When it comes to the storage of data generated throughout the system, the choice of the particular data storage type to use is mainly determined

by both the type of data to be stored and the how and where in the system, the data is accessed in future.

The critical determining factor in determining the appropriate data storage solution is ensuring it can adequately satisfy the entire set of system's non-functional requirements. In particular, it is paramount that the specific data storage system used is fault-tolerant and scales horizontally.

The storage types in the proposed system can be consolidated into three major categories, as outlined below:

1. NoSQL Document-oriented Data Stores,
2. Time-series Data Stores and,
3. Distributed Block Storage.

The inclusion of several data storage classes in the system design is portrayed in Figure 4.2. The particular use case of each data storage type is addressed in what follows:

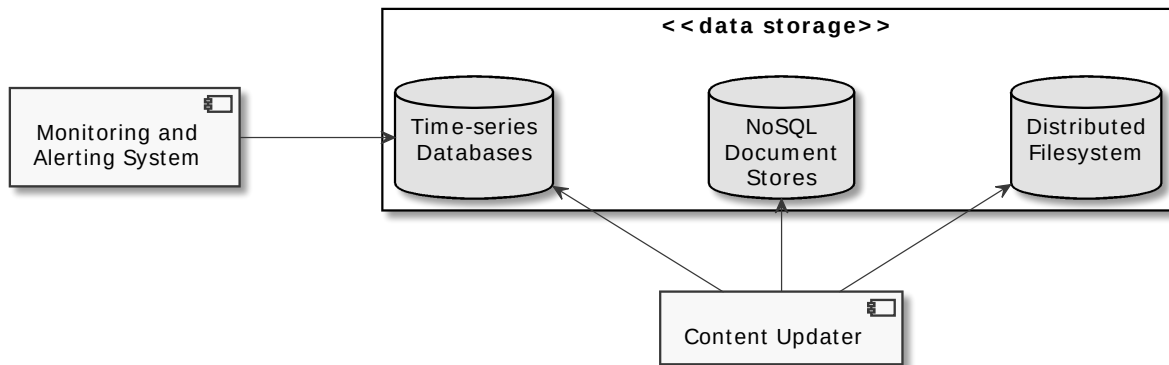


Figure 4.2: Proposed System's Logical Design of Data Storage Component

- **Time-series Data Stores** The data produced by each of the system components' are, to no small extent, time-series in nature. In other words, they constitute events that have occurred at a certain point in time or are measured quantities that vary as a function of time. Hence, each of the data points always has a timestamp associated with it that represents the particular point of time, a particular quantity has been measured, or an event of interest has taken place.

A time-series database is optimized to store thousands of data points where each point has an associated timestamp with it. The database swiftly makes the data available upon receiving a query for the data pertaining to a particular time interval. Therefore, a time-series database is merely a database with an optimized design to efficiently handle timestamped events and metrics. Lastly, it is worth noting that the above timestamps' format follows the standard RFC3339 [32] protocol.

- **NoSQL Document-oriented Data Stores** The proposed system primarily makes use of the NoSQL databases to store the application-wide configurations and settings.

A NoSQL database can be further classified into several categories, including but not limited to, the key-value stores, the document stores, the column-oriented databases, and the graph databases. Each database category is optimized for one particular type of problem, and therefore, its particular preference is depended upon the specific requirements of the use case.

As previously discussed, the NoSQL data store component of the proposed system is provisioned to store both the past and current application-wide configurations and settings. As such, the specific type of the NoSQL database used herein is the document-oriented data store, which allows for the storage of semi-structured JSON¹-like documents that are composed of several keys and corresponding associated values, each having either a flat or hierarchically nested organization.

NoSQL data stores, as opposed to the relational databases, do not require the data to conform with a particular schema or otherwise, possess a specific tabular relationship. In contrast, the data can be stored as any set of key and value pairs with no constraints on the data schema or requirement to match a predefined structure. The latter makes the NoSQL databases expressly fitting for applications that involve manipulating broad sets of semi-structured data.

Whereas there is no schema requirement imposed a priori on the data being stored, schemas can nevertheless be enforced wherever conformity with a particular data schema is desired or required per the application requirements.

¹JavaScript Object Notation

As noted above, the NoSQL databases' advantages include simplicity of design and increased flexibility in the migration of data from one database system to another. The more straightforward design pattern thereof empowers the seamless horizontal scaling of the databases to several machines in a computing cluster.

Similar to the relational databases, a large proportion of the NoSQL databases do provide mechanisms for accomplishing replication and fault-tolerance. Nevertheless, the attributes and properties of such replication have fundamental differences with that of a conventional one.

In order to further clarify the distinction, the relational databases are commonly designed such that a *strong consistency* among replicas can be achieved. In contrast, the replication in a NoSQL database follows the concept of an *eventual consistency*.

Strong consistency, also referred to as immediate consistency, guarantees that an update to data is instantly propagated to all the other existing replicas. As such, when queries are made against the data, always the same result is consistently returned regardless of the particular replica used.

In general terms, the above requires that first, a particular update be guaranteed as having been successfully propagated to all the other existing replicas. Consequently, an update to data will form part of the returned results of a query, exclusively after the latter data update has successfully concluded.

Intuitively, the synchronization and coordination required among the replicas along with the corresponding overhead incurred as a consequence may severely impact the scalability of large-scale data-intensive applications.

However, such a strong consistency is not usually required for the big data applications in particular, given that in such cases, an eventual loss of one or more data points in large volumes of data is far more permissive and justifiable, especially in light of the superior scalability and performance that is achieved as a result. As such, for a large majority of the NoSQL databases, the data replication follows the concept of eventual consistency as opposed to a strong one.

The precise methodologies used to achieve an eventual consistency fall outside the scope of this work. However, it should be noted that in an eventual consistency regime, data may still be queried from any of the available replicas. Nonetheless, there is no guarantee that the query results returned by different replicas will be consistent with each other. However, absent an immediate update to data, all the replicas will eventually be consistent as the updates are eventually propagated.

- **Distributed Block Storage** A highly scalable and distributed filesystem is needed to enable the seamless distribution and replication of data blocks and files across the entire cluster, thereby making them accessible to all nodes that make part of that cluster. The proposed system requires a distributed filesystem, principally for distributing and replicating the container volumes in the cluster.

As described earlier in Section 4.2.2, a container can be regarded as a process that runs in an isolated execution environment. Such an environment may hence be customized to include the desired set of software packages and configuration files. As a matter of practice, after the latter step's changes are complete, the containerized process is therefore extensively tested to ascertain the created environment satisfies all the process requirements.

Therefore, once the above requirements are satisfied, an image of the entire environment is created. The image is therefore used for the deployment and shipping of a container to the cluster nodes. The running image serves as a blueprint to create new running containers on each one of the cluster nodes.

Intuitively, the whole process is repeated for every new release of the containerized software. A trivial yet significant consequence of such an approach is that a required change to the environment or the software code, may be applied once and hence replicated at scale to any other running container thereof.

Understanding the above process is essential to motivate the use of the distributed file system in the proposed system design. Running containers are created from their corresponding container images as noted above. Therefore, upon the creation of a container image, in the absence of a strategy to keep the individual configuration files and data within a virtual execution environment of a container, the entire environment will be

inevitably replaced by the new environment used in the recently created image. In other words, the data is not kept across the container updates.

While this is indeed the intended outcome for most of the containers' data, certain data and configuration files still need to be persisted and hence re-used through container updates.

As such, in order to persist a container's data, the volumes from the host are mounted under specific paths in the container's underlying filesystem directories. Therefore, any data generated or residing under those specific paths of the container's filesystem, will in fact be stored outside the container environment and on the host. Consequently, the container data may be safely kept intact across any future updates that is made to the containerized process.

The persistence of the container data on the host's filesystem enables the seamless execution of new containerized software or process releases. On that account, it is far more advantageous to persist the container data in a distributed filesystem that is accessible by all cluster nodes as opposed to the host's filesystem. As the reader may anticipate, the latter means that a container can now transparently be relocated or spawned on a different node in the cluster, with absolutely no interruptions or need for reconfiguration.

Likewise, the fault-tolerance and failover of the proposed system's containerized services is also made possible through replicating the containers' data in the distributed storage accessible by all nodes. Therefore, in the event of a node's failure, the executing containers can immediately be launched on any of the other working nodes in the cluster.

4.2.6 Logical Design of Video Analytics

The proposed system's video analytics component is designed to perform the required video analytics tasks on the system's video input streams. The component is designed from scratch in an extensible manner that supports the onboarding of any additional video analytics algorithm. Moreover, two state-of-the-art vision algorithms aimed at performing intelligent object detection and classification on the video frames, as well as tracking a set of objects of interest as they move within the regions monitored by a camera, are incorporated into this component. The

functions accomplished by each of the before-mentioned algorithms will be further described in Section 4.3.5.

Moreover, the component is implemented such that it may perform the video analytics jobs on the input video frames in a parallel fashion. To that end, the component uses multiple video analytics worker threads, each working on one of the video frames extracted from the input video stream. The latter approach is employed in the devised video analytics component to allow for maximizing the overall system's throughput and thus, make a more efficient use of the underlying computing resources required for the carrying-out of the video analytics job.

4.2.7 Logical Design of Data Pre-processing and Enrichment

The previous section described the large variety of collected metrics and statistics across different components of the cluster. Several functions of the proposed system require the persistence of the entire set of collected metrics in long-term storage.

In some instances, the same data scraping tool may well accordingly apply any required transformation to data and place it on durable storage. However, that primarily depends on the type of data being collected and the particular long-term storage type used to persist data. However, as described above, the video analytics application's runtime metrics, data, and results are streamed in real-time and placed on different data streaming system queues.

Consequently, the streaming data above is later consumed by different workers to *a)* apply any required transformation of data and *b)* enrich the retrieved data by either adding new fields or aggregating data from other streams. The enriched result is ultimately forwarded to the persistent storage for long-term storage and reuse.

4.2.8 Logical Design of Data Security

The proposed system's Data security component is concerned with both securing the data in transit and safe-guarding the node's services against unauthorized access. The former is achieved by encrypting the communications among the nodes so that no sensitive information is sent or received in plain text and hence, not exploitable by potential eavesdroppers or malicious software.

On the other hand, the latter is achieved by implementing appropriate packet filtering rules and technologies to restrict and drop communications from hosts other than the participating nodes of the deployed cluster used by the proposed system.

4.2.9 Logical Design of Presentation

As discussed in Section 4.2.4, an extensive set of monitored metrics and statistics is collected at different system layers and is stored in time-series databases. However, data visualization tools are required for real-time monitoring of data and interactive exploration and analytics thereof.

The visualization tool connects to the storage system where the collected metrics reside and therefore, visualizes any desired set of metrics in real-time using different plots and graphs. An important design constraint is that the particular visualization tool used must swiftly integrate with the other time-series databases used in the proposed system’s implementation.

4.3 System Physical Design

4.3.1 Physical Design of Service Orchestration and Management

As discussed in Section 3.5.1, all the proposed system components are implemented using containers. That is done to enable consistent development and deployment. This work uses Docker [33] technology for containerization of all the system required components. The orchestration and deployment of containers are achieved using the Docker’s native clustering engine, *i.e.*, *Swarm*.

Docker Swarm creates a centralized view of the cluster and provides a scalable and flexible mechanism for the management, configuration and orchestration of the containers throughout the cluster. To that end, first a Swarm cluster is created on one of the system deployment’s participating nodes. Subsequently, the remainder of the participating nodes are joined to the Swarm cluster using the secure token generated upon the creation of the Swarm cluster.

Computing hosts that join a Swarm cluster are referred to as nodes in Docker terminology, and the same terminology is likewise adopted throughout this work. In general terms, each container has one specific goal and one set of actions it performs and functionalities it provides. The containers can be grouped together to create a more complex but yet well modular application. In Docker’s terminology, the latter is referred to as a *stack*.

Services deployed across the nodes of a Swarm cluster, communicate with the other system components or external services using either the host's network or an overlay network that is created on top of the host network and permits a group of inter-dependent services to connect with one another.

The containers running on Docker can be configured to use the host's network. However, in such a case, both the container and the Docker host will be sharing the same network and thus, there will not be an isolation of network traffic among them. To circumvent that, Docker may be configured to run the containers in an isolated bridge network that is provisioned on the Docker host.

The software bridge network above, allows connectivity among the containers connected to the same bridge network while still isolating and restricting any incoming traffic from containers outside to the bridge network. Docker achieves this by automatically generating and adding any corresponding and required set of firewall packet-filtering rules on the host.

External reachability of services deployed on the Docker local bridges is achieved by exposing and publishing the required TCP/UDP ports of the service on the host and thus, forwarding all the incoming traffic destined at the exposed port on the host, to the corresponding service running on the host's local bridge. The above networking scheme is suitable when both the services and the containers dependent thereon are deployed on the very same computing host.

However, the networking mode of choice for the service stacks in Docker is that of the overlay network. An overlay network provides connectivity among all services pertaining to a Docker stack, regardless of where in the Swarm cluster the particular service is exactly deployed.

Moreover, creation of layer two connectivity among services through the overlay networks in a Swarm cluster has several advantages, including ease of configuration, name resolution and service discovery among the containers in a stack. By default, the overlay network traffic is not encrypted.

The lack of encryption in and of itself does not pose a risk to data's security, for as long as the communicating system services employ a secure channel of communication (such as a TLS² session). However, if sensitive data is to be transmitted in plain text, the overlay network can be configured to encrypt traffic using AES [34].

²Transport Layer Security

4.3.2 Physical Design of Data Streaming and Message Queues

Apache Kafka [35] is an open-source and distributed stream-processing platform that aims to provide high-throughput and low-latency handling of massive real-time data feeds and was first introduced in [36].

Kafka is distributed and a Kafka cluster is created by launching one or more Kafka brokers across a set of servers. Apache ZooKeeper [37] serves as a centralized service for the reliable and distributed coordination of Kafka brokers.

To discuss how Kafka serves to carry out the proposed system requirements, the notion of topics, partitions, messages and consumer groups in Kafka must first be introduced.

Kafka uses *topics* to store and stream messages of a particular type. Each topic is composed of one or more *partitions*. As the name suggests, each partition at a high-level, contains a subset of the overall messages pertaining to a particular topic. A *message* in Kafka is simply a stream of bytes. A byte-like representation of a data structure or dataset can be created by serializing the data using one of the existing encoding schemes.

In Kafka, a *consumer group* can be described as the set of one or more consumer instances running in parallel and all forming part of a single subscriber. The parallel consumers in a consumer group—likely running concurrently—each consume a subset of the full set of messages belonging to a topic. The latter subset, is one of the many data partitions that may constitute a Kafka topic.

Publisher-subscriber or Point-to-point Messaging Pattern

Kafka can be configured to either facilitate a Publisher-subscriber messaging pattern among services across the cluster or instead, enable high-throughput and low-latency data exchange between a couple of services, each serving as one endpoint of the communication.

However, the foregoing are not mutually exclusive. As discussed later on in this chapter, both cases' advantages can still be achieved through a careful and meticulous setting of Kafka's configuration parameters.

A copy of data produced by a publisher under a topic name is therefore ingested by several consumers who subscribe to that particular topic in a publish-subscribe scenario. Each subscriber

will thus receive a full copy of the messages published to the topic.

Such a setup has all the advantages of a publish and subscribe pattern, yet in and of itself cannot fully take advantage of a Kafka cluster to maximize the throughput and cut down on the data transfer latencies.

The latter, on the other hand, follows a point-to-point delivery model. In a point-to-point delivery scheme, the throughput is instead maximized by employing multiple consumers in a consumer group. The latter set of consumers can be configured to work concurrently in parallel to collaboratively ingest real-time data feeds; however, provided that the particular topic's data is split into at least more than one partition.

As such, a consumer in the consumer group will retrieve just a subset of the entire set by ingesting the existing partitions' data. A direct consequence of that which is worth considering is that the maximum number of consumers in a group must be configured to be at most equal to the number of existing partitions. The allocation of a larger number of consumers in the group will invariably result in additional consumers spend time idly starving for data.

Load Balancing

Kafka's choice of partitioning the data early on in the process has several advantages. First, it effectively removes all the operation and maintenance overheads and complexities associated with the simultaneous consumption from a single data store. Absent a data partitioning strategy, the system's faultless operation would have required a perfect synchronization among the consumers and detailed tracking of all the transactions performed by each consumer.

Second, it serves to facilitate the balancing of the load in the cluster by pervasively distributing both the data partitions and the consumers in a group, all across the cluster. Hence, the partitions in Kafka may be regarded as the most primitive units of parallelism. Using a sufficiently large number of partitions increases the maximum achievable throughput while also resulting in a better distribution of load in the cluster.

However, an excessive partitioning of data can negatively impact the end-to-end latency and lead to low resource utilization. Therefore, the overheads associated with running multiple consumer processes may not fully justify the scant gains that are consequently achieved in the throughput.

The impact on the end-to-end latency is consequent to the fact that Kafka internally does not expose a message for consumption unless and until the latter message has already been replicated to all other in-sync replicas of that partition.

The rationale for Kafka’s employment of such an approach is to guarantee that in an eventual failure of a Kafka broker, the replicas of data used for fault-tolerance and resiliency in the cluster do never end up being in an inconsistent state with the primary replica serving as the leader of a particular partition.

Latency and Throughput Trade-offs

As discussed in the section above, the data throughput in Kafka can be maximized through data partitioning. However, Kafka can still deliver significantly high throughput with a considerably low end-to-end latency even when data is not partitioned.

The above consideration is crucial, as there are specific design requirements wherein the partitioning of data may thus hinder our ability to satisfy those requirements. For instance, it should be noted that Kafka can guarantee the ordering of messages only within a single partition and hence not across the several partitions that make up a topic.

The choice of multiple partitions can likely impede or at least hinder the possibility of achieving a total ordering of the messages retrieved from a Kafka cluster. The latter is essential wherever a system design requirement mandates that messages of a particular type be delivered in the same sequence as they arrive.

To further clarify, such a design requirement does not fully impair the practicability of a solution that uses multiple partitions in an effort to maximize the throughput. However, it adds to both the system’s complexity and end-to-end latency. The increase in complexity is due to the consequent need to devise additional system components that can aggregate and buffer the messages received by all the consumers of a group, thereby performing a total or partial sorting of messages.

Negligible Slowdowns Compared to In-memory Data Stores

In-memory storage is perhaps the most widely adopted strategy to achieve significantly low latency for data access. The I/O operations are performed hundreds of times faster in-memory

compared to a conventional disk. Even when a traditional disk is replaced by a modern Solid State Drive (SSD), the I/O rate achieved is still dozens of times inferior to that of memory [38].

For a traditional disk, the most significant factor contributing to the experienced I/O delays is the *seek time*, *i.e.*, time taken for the disk controller to locate the particular chunk of data and move the read/write head to that specific position.

Therefore, keeping data in memory is an exceptionally effective way to achieve low latency in retrieving data and significantly diminish the I/O latencies incurred in data-intensive operations. Nonetheless, the several advantages of fast memory are often hindered by its relatively expensive cost and comparatively limited storage capacity.

The latter limitation is often mitigated by frequently evicting in-memory data older than a certain threshold, typically in tens of seconds.

Systems employing such a strategy operate under the assumption that in a real-time data application, data chunks' significance and value are progressively diminished as the data ages.

While this is indeed a valid assumption for a wide range of real-time applications, in the event that an application still requires access to the recently evicted data, the above scheme could negatively affect the robustness fault-tolerance of the application.

This may be mitigated by employing redundant readers of the published data, however, with the cost of adding more complexity to the system. Besides, to ensure data consistency, the replicas additionally need to coordinate and reach consensus on their internal state, therefore, further complicating things.

As such, the relatively low inflation in the end-to-end latency can reasonably be justified by the commensurate gain that is sought through the actual persistence of the real-time data feeds for arbitrarily more massive amounts of time.

Kafka does not rely on the memory to perform fast data operations. Instead, it aims to minimize the disks' seek time by performing sequential reads and writes to the disk, thereby effectively mitigating the incurred seek times. Long seek times occur when the storage blocks making up the fetched data are not all stored in contiguous blocks of the underlying file system.

The specific method used by the underlying filesystem to store data across blocks is outside this work scope. However, it should be noted that the operating system will first spill the data in as many contiguous blocks as it can allocate. If more blocks are required thereafter, the

operating system allocates the remaining blocks from the free storage regardless of where those blocks are physically located on the disk.

However, by performing sequential I/Os, the experienced latency in Kafka is significantly reduced and brought as close as latency exhibited by an I/O operation in memory in certain circumstances [39].

Kafka’s choice as a framework to handle all data-intensive operations involved in collecting and distributing real-time data feeds, in the proposed system’s design, is therefore further justified with due consideration of the above.

Fine-Tuning Kafka

The previous section thoroughly discussed the several factors impacting the real-time throughput and end-to-end latency of data feeds in Apache Kafka and described how the proposed system mitigates or leverages them depending on the case. Throughput and latency in Kafka are two competing factors wherein the gain in one is inevitably accompanied by a commensurate loss of the other. Therefore, a trade-off between the throughput and the latency is required.

Depending on the application’s specific requirements and its particular implementation, a nearly perfect balance between throughput and latency can be achieved by appropriately tweaking specific significant yet less often contemplated Kafka configuration parameters, *i.e.*, **Linger Time** and **Batch Size**.

Before elucidating what each of the above parameters represents, a more in-depth look into the actual process followed by a Kafka producer in forwarding and therefore committing the intended messages to a Kafka broker is needed. In a Kafka producer, the transmission of messages to a Kafka broker does not typically initiate immediately upon the reception of a message. Instead, the producer queues up the messages to be transmitted in a buffer and will only initiate the transmission if the buffer is full and cannot accommodate any more data.

The strategy mentioned above allows for several messages to be grouped together in batches, thereby maximizing the throughput by having a much larger payload of bytes to transmit each time a connection is established with the broker. Moreover, larger payloads in transmissions also result in larger sequential writes to the disk and, consequently, an increased disk throughput on the broker side.

The **Batch Size** parameter is used to specify the buffer’s length and can be configured to be of any size depending on the system’s specific performance requirements. However, the end-to-end-latency of a Kafka broker is increased when data is buffered and not immediately transmitted.

The **Linger Time** parameter in Kafka allows for an upper-bound setting on the amount of time a producer may spend idle lingering for the buffer to be filled before initiating a transmission. When the predefined threshold specified in **Linger Time** is exceeded, the producer will initiate the buffered data transmission regardless of the buffer’s capacity used.

As the reader may anticipate, the linger time is, in most cases, configured to be almost as large as the maximum latency the system can safely tolerate while still meeting the pre-established system requirements. On the other hand, the batch size should be as large as possible; however, with due consideration of the available computing resources in the underlying system where the producer executes.

4.3.3 Physical Design of Monitoring and Alerting System

A block diagram for the system monitoring toolchain used in the Monitoring and Alerting system’s physical design is illustrated in Figure 4.3. The description of each constituting part and a discussion of the specific role each has in the overall system design is addressed in what follows.

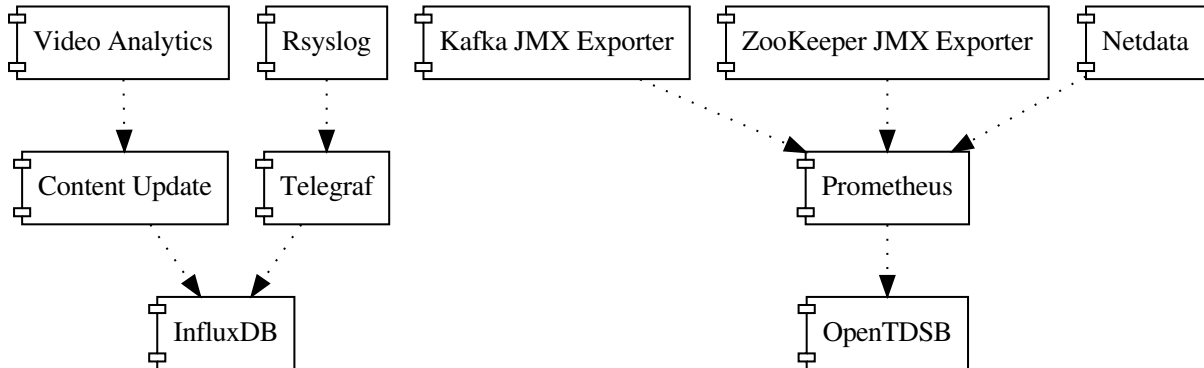


Figure 4.3: Proposed System’s Monitoring and Alerting Toolchain

Netdata and its Role in the Proposed System Design

Netdata [40] is a powerful and open-source metrics collecting engine that seamlessly runs on a physical or virtual machine. Netdata can be used to collect hundreds of performance metrics from the computing resources and the underlying hardware. Netdata has the additional advantage of providing real-time visual insights into the performance as it employs a variety of intuitive charts and plots which may be used to examine and query data interactively.

Netdata continuously monitors a resource for a predefined observation period and thus produces a measured value for that interval. Moreover, Netdata, under the hood, intelligently uses a set of predefined rules to aggregate a variety of metrics from different primary subsystems of the executing node, thereby automatically providing the user with high-level and concise performance recommendations and alerts.

Additionally, Netdata can be configured to instantly trigger an alert based upon a set of customized and predefined rules on the collected system metrics. Therefore, Netdata is a notable addition to the proposed system's monitoring toolchain.

The above system-level metrics are collected from all the nodes across the cluster. As such, Docker Swarm is configured to deploy the Netdata service as a global service, *i.e.*, a service replicated across every node that forms part of the Swarm cluster.

Persistence of Collected Metrics

Netdata performs the real-time collection of performance metrics and uses the system's memory for the temporary storage of the collected metrics by default.

Therefore, the gathered metrics are not persisted on disk and are instead kept in memory only for as long as the user is interactively exploring data. This is partly due to the abundance of metrics being monitored, and therefore, the storage requirements imposed, which depending on the particular use case, may or not be desirable.

Nonetheless, the interactive exploration and analysis of large datasets often require that data be kept in memory so that it may be accessed and retrieved reasonably fast.

The persistence of the gathered metrics can be merely achieved by spilling the large volume of collected data to disk. However, this can lead to poor response times for the interactive

queries later made on the persisted data.

In the proposed system, we would ideally want to have the totality of collected metrics throughout the systems all stored in a virtually centralized system, where it may later get queried and explored interactively.

Prometheus and its Role in the Proposed System Design

Prometheus [41] is another open-source monitoring solution that can collect statistics from various metric collection agents and applications. The process is often referred to as Data Scraping, where a process extracts and imports the data, usually in a human-readable format that is produced by another program.

Prometheus scrapes data from multiple sources, indexes them by a collection timestamp and then ultimately persists them in a specialized time-series database so that it may be quickly and swiftly retrieved in future.

As discussed above, Netdata is set to expose the metrics it collects and thus make them all available through REST³ API⁴s. As such, Prometheus is configured to make periodic API calls on predefined intervals to each of the Netdata instances that run in the cluster. Hence, Prometheus performs data scraping and retrieves the most recent measurements made by each Netdata instance.

Telegraf and its Role in the Proposed System Design

Telegraf [42] is a system monitoring and metric collection tool that performs the other monitoring and data scraping functions required by the proposed system. The choice of Telegraf as an additional system-wide performance metric collector is made due to several reasons briefly discussed in what follows.

Motivation for Use of Multiple Data Scrapers

First of all, an additional system monitor complements the set of collected metrics by collecting any other metric not monitored by the first system monitor, making up for gaps in the first

³Representational State Transfer

⁴Application Programming Interface

collected set.

Second, it provides a feasible way to create redundancy for the collection of metrics, which is essential to guarantee the proposed system’s correct functioning and ascertain the robustness and reliability of the alerting system, operation of which is heavily reliant on the collected metrics.

Hence, it is crucial to ensure that all system components are continuously and persistently monitored, to the most considerable extent possible.

Instrumenting Kafka and ZooKeeper: Motivation and Discussion

The detailed and granular resource utilization statistics collected from nodes and containers throughout the system are typically sufficient to monitor the overall systems’ performance and, more importantly, allow for proactively triggering the required mitigatory actions based on the generated alerts.

Nonetheless, an even more granular collection of metrics and key performance indicators is desired for certain system components, such as the message brokers.

The Kafka message brokers used in the proposed system message queues component play a crucial role in facilitating the inter-component communication and data exchange. As such, the data exchange statistics, such as the producer production rates, consumption rates or consumer lags, are of significant value and must preferably make part of the superset of collected metrics throughout the cluster.

Therefore, in this work, both the Kafka brokers and the ZooKeeper are instrumented to collect and expose a broad set of counters and data fields, thereby shedding light on their internal runtime performance, including but not limited to granular statistics on the status of active topics, producers and consumer groups.

Both Kafka and ZooKeeper are essentially Java programs that execute in a Java Virtual Machine. As such, traditional Java tools used to manage and monitor applications and system objects can likewise be used to scrape runtime statistics for each of the components, as mentioned above.

One such Java monitoring tool is the Java Management Extension (JMX) [43], and it provides several probes to instrument a Java application and performs in-depth monitoring of

its performance throughout the execution.

Therefore, Apache Kafka and Zookeeper instances are instrumented with the JMX Exporter, which effectively collects a rich set of runtime statistics from each process and hence, exposes the collected metrics through REST APIs.

As such, the set of provisioned data scrapers in the proposed system make frequent and regular API calls to each of the above components in order to extract and collect the set of exposed statistics thereof.

Scraping of Logging Information

As described earlier, the logging information produced by different system components is collected, streamed in real-time and ultimately stored in the persistent storage for long-term use.

While each component may comprise a distinctive stack of software packages, the logs generated by each system component still follow the same standard format. Therefore, it is now feasible to pervasively and extensively collect and analyze all the logging and debugging information generated by each system component.

The standard message logging format used for logging is System Logging Protocol or Syslog for short, as specified in RFC3164 [44,45]. Adopting a standard protocol for logging effectively decouples the particular software that generates the logs from other system tools used to read and analyze the logs. Hence, a large majority of software packages use the Syslog protocol to produce debugging and logging information.

The logging information originating from each system component is typically stored as a textual file somewhere in the component host's underlying filesystem. Hence, exposing the logging information to the data scrapers throughout the system necessitates the provisioning of a specific set of processes mainly responsible for locating, collecting, and exposing the totality of the logging information via well-defined and accessible APIs to the data scrapers.

To that end, Rsyslog [46], an open-source utility, is used to practically collect and, therefore, expose the log messages over the network.

Collection of Video Analytics Job Results and Runtime Statistics

Both sets of video analytics runtime metrics and application-wide analytics results originate from the proposed system's devised video analytics component. The component mentioned above is instrumented to consolidate each of the collected metrics into a corresponding set and, therefore, publish each set to a corresponding topic on the Kafka broker.

In this case, the devised content updaters described in Section 4.2.7 serve as the actual data scrapers that populate the collected data to each of the corresponding data stores.

4.3.4 Physical Design of Data Storage

The physical design of the data storage component of the proposed system is discussed in this section. The primary rationale behind the choices made herein is ensuring that each data storage can meet the proposed system's non-functional requirements and further provide interfaces for integration with the other system components. Figure 4.4 depicts a high-level overview of the physical design of the proposed system's data storage component. A detailed description of each constituting part, along with the primary roles each play and reasons for the integration of each, is discussed in what follows.

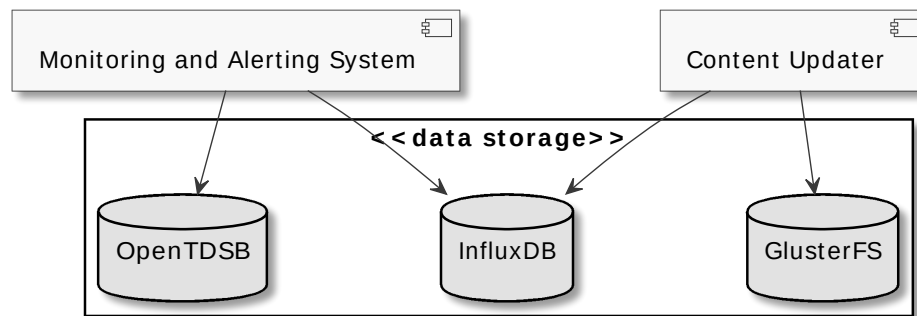


Figure 4.4: Proposed System's Physical Design of Data Storage

NoSQL Document-oriented Data Stores

MongoDB [47] is used to store both the past and current application-wide configurations and settings. MongoDB is an open-source document-oriented database. MongoDB satisfies the proposed system requirements for document-oriented storage, given that it natively supports

the creation of highly-available, distributed and replicated JSON-like documents.

High-availability is achieved through replication. MongoDB uses the notion of *replica sets*, which are essentially two or more copies of data spread across different nodes. Only one of the replicas is used for reading and write operations.

However, if the node serving the replica set fails, an election process is automatically launched to select the next leader for the particular replica set. However, as it was thoroughly discussed in Section 4.2.5, in such a scenario, the newly elected leader for the replica set will eventually be consistent with its predecessor.

Distribution and load balancing is achieved by *sharding*, *i.e.*, horizontal partitioning of data. Optimally, data among shards are evenly distributed, and each shard is physically located on a different computing node. Consequently, load balancing is achieved by serving the data read requests from multiple physical servers.

Time-series Data Stores

The required time-series data stores for the physical design of the system are briefly described in what follows:

- **InfluxDB:** One of the time-series databases used in the physical design of the proposed system is InfluxDB [48]. InfluxDB is open-source and is optimized for fast and swift retrieval of time-series data.

The database will be used for the real-time storage of collected system-wide performance metrics and the results of video analytics jobs performed on each of the input video streams. Additionally, the database serves as a storage system for the totality of runtime logs collected from all the nodes and containers in the system.

The choice of InfluxDB is mainly due to its high-availability features and its swift integration with other system components, responsible for both collection of metrics and the interactive analysis and visualization thereof.

- **OpenTDSB:** The time-series database used for the storage of the scraped data by Prometheus is OpenTDSB [49]. OpenTDSB is another open-source and highly distributed

time-series database. OpenTDSB runs on top of HBase,⁵ an open-source non-relational distributed database that, in turn, runs on top of HDFS⁶ distributed storage.

The choice of openTDSB as the storage backend for Prometheus is mainly due to its ability to index and persist relatively large volumes of scraped data at considerably high rates (typically every few seconds), thereby enabling data to be swiftly queried, graphed and analyzed.

Distributed Block Storage

GlusterFS [50] is used to facilitate creating a distributed filesystem to spread the data over different nodes in the cluster and therefore make it accessible to all cluster nodes. GlusterFS is an open-source and scalable distributed filesystem that can aggregate the available disk storage on the cluster nodes into a single unified global namespace.

Moreover, GlusterFS may be configured to create distributed and globally replicated volumes throughout the cluster. Intuitively, in the latter case, the distributed volume capacity cannot exceed the disk storage capacity of a node.

However, global replication of files means that any file placed on the distributed filesystem is replicated in a distributed fashion to all the cluster nodes' local disk drive. Consequently, in addition to greater reliability and availability for data, the nodes can achieve superior read performance when accessing the distributed data.

4.3.5 Physical Design of Video Analytics

The developed video analytics application comprises multiple workers at a broad level, each performing a set of pre-established tasks to accomplish the specific objectives assigned to in the design stage. The mode of execution, *i.e.*, parallel or sequential execution, varies between each of the above workers and is determined by the particular functions they perform.

The specific degree of parallelism used for each class of worker, *i.e.*, the number of workers executed concurrently in parallel, depends on the computing task each must perform and the

⁵Apache HBase™ is the Hadoop database, a distributed, scalable, big data store. See <https://hbase.apache.org/>

⁶Hadoop Distributed File System (HDFS™): A distributed file system that provides high-throughput access to application data. See <https://hadoop.apache.org/>

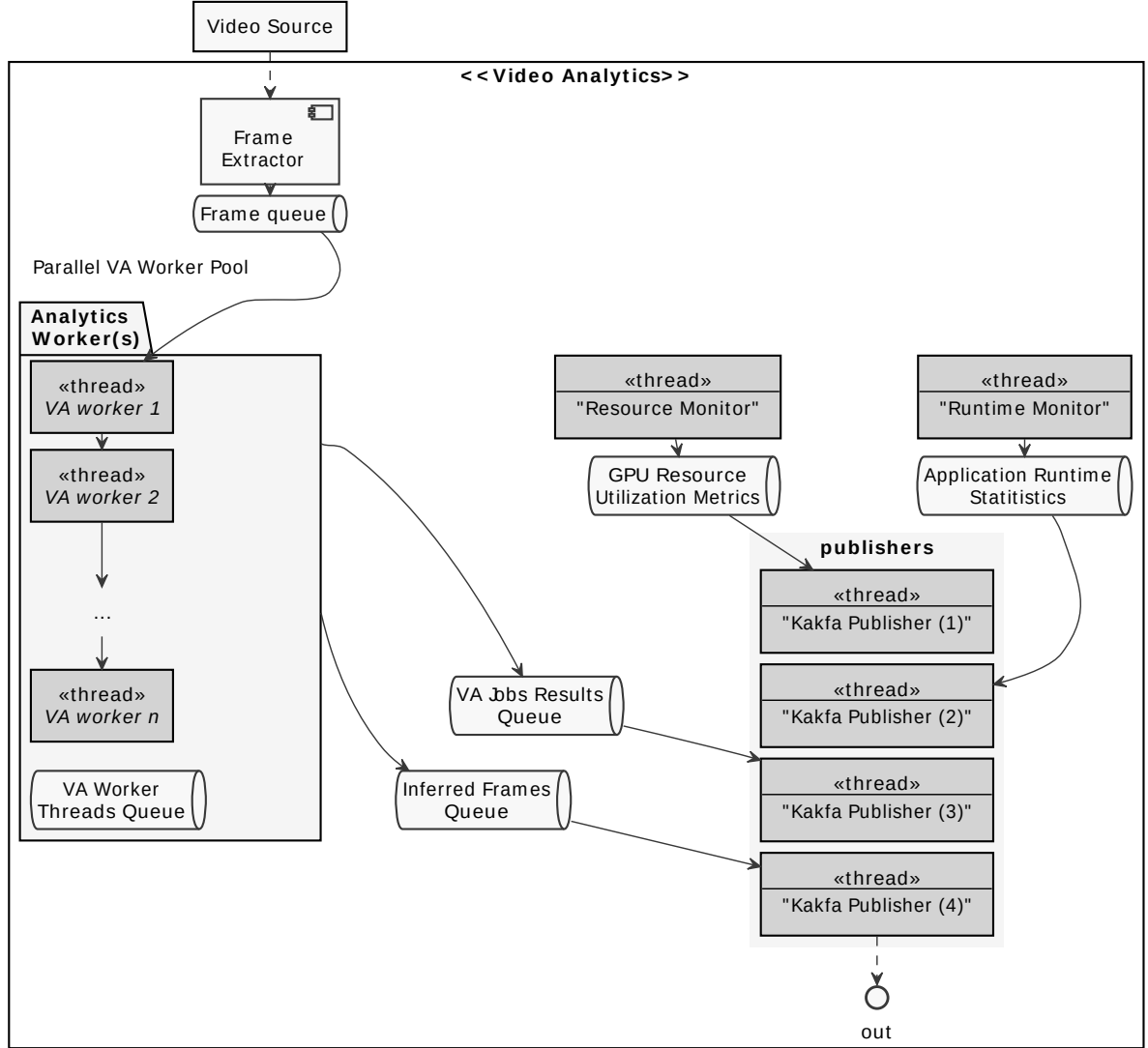


Figure 4.5: Video Analytics Component Block Diagram

specifics of the underlying hardware where each executes. Consequently, the degree of parallelism is determined in the proposed system's deployment and evaluation stage.

Moreover and complementary to the workers mentioned above, the devised video analytics component uses several synchronized and thread-safe queues to enable message passing and communication among the workers that comprise the video analytics component.

Worker Threads The constituting workers of the video analytics application perform a variety of functions and can be consolidated into several classes based on the specific functions each performs, as outlined in what follows:

- **Frame Retrieve and Extract** Primarily responsible for establishing a connection with the camera or any other real-time video source to retrieve the video streams and accordingly extract the corresponding video frames. The OpenCV [51] library is used for the retrieval of frames from the remote video source. OpenCV is a feature-rich library used for machine learning and image processing and is primarily developed to enable the optimized execution of computationally-intensive vision algorithms.
- **Video Analytics (VA) Workers** The video analytics workers execute various computer vision algorithms on the extracted video frames. The outcome contains insightful information and knowledge that is extracted and inferred from a single video frame or a sequence thereof.
- **Runtime Monitoring** Runtime monitoring workers collect application-level performance metrics on the runtime that are, in part, essential for the correct functioning of the system's monitoring and alerting component. The collected metrics are used to devise and trigger appropriate mitigatory actions when the application undergoes performance degradations or failures during the runtime.
- **Resource Monitoring** The resource monitoring workers collect detailed hardware utilization statistics from the underlying resources that are otherwise not monitored by other system components. The latter applies explicitly to compute resources such as GPUs, monitoring of which often necessitates vendor-specific tools.

The resource monitoring worker makes a measurement every fifty milliseconds and stores the measurements in an in-memory queue. Therefore, the Runtime Monitoring worker accesses the aforementioned queue in memory each second and collects all the recent measurements thereof. Once collected, then it makes an average of all measured values and places the result back in the queue. Therefore, the result is marked as publish ready, so that it may be published to the message queues for distribution to respective consumers.

- **Kafka Publishers** These workers are created to enable fast and real-time streaming of the whole set of video analytics job results and collected metrics throughout the devised video analytics application's execution. Consequently, the data publishers stream the

collected data to the corresponding message queues, each of which is in turn, subsequently accessed by the respective set of consumers of that data.

Thread-safe Queues The several queues of the video analytics component contain and hold different sets of data required for the correct functioning of the application and can be broadly consolidated into two major categories, outline in what follows:

- Thread-safe queues implemented to facilitate inter-worker communication, hence, enabling data exchange among the workers constituting the application.
- Other thread-safe queues implemented to be used by each of the above workers to store the data resulting from the particular processing or collection task performed by each.

The above queues may comprise the raw video frames extracted from a video source or contain visually annotated and processed frames based on the information derived by executing the video analytics jobs on each frame. The latter annotations on the frames help provide the system's end-user with a quick glimpse of insight into the data that is derived through inference, using the vision algorithms.

On the other hand, the rest of the above type queues may contain granular or aggregated data produced by other components that constitute the developed application.

More specifically, the following thread-safe queues are built into the video analytics component:

- **GPU Resource Utilizations Queue** The queue contains the collected GPU resource utilization records. As discussed earlier, monitoring certain hardware resources such as GPUs may require vendor-specific tools.
- **Application Runtime Statistics Queue** The application-level debugging and performance statistics are collected during the runtime and placed on this queue, prior to them getting published to the message queues. The set of the runtime metrics is collected at every one-second interval throughout the execution of the program.
- **Video Analytics (VA) Job Results Queue** The results obtained through the execution of the video analytics algorithms on the input video frames are collected and placed in

this queue for transmission. These include for instance, the set of detected objects and their corresponding object class.

- **Frame Queue** The frame queue holds the set of video frames that are extracted from the real-time video streams input to the system. The VA workers access the queue upon getting spawned to retrieve a frame, on which the video analytics job is performed.
- **Inferred Frames Queue** The results of the video analytics jobs can be used to visually annotate the video frames for instance, with bounding boxes surrounding the detected objects. OpenCV library is used to perform the latter operation on the frames. The resulting frame, referred to as an inferred frame herein is placed on this queue. To that end, the bytes-like representation of the inferred video frame is created using the base64 encoding scheme and the encoded string is placed on the above queue for distribution to other corresponding system components.
- **VA Worker Threads Queue** This queue is of finite size and contains references to the set of active VA worker threads. The queue is constantly polled to probe the health of executing VA worker threads and launch of new workers as necessary.

AI Engine Tensorflow [52] is an open-source and scalable machine learning platform and provides several high-level libraries for the execution of a wide range of artificial intelligence and deep learning algorithms and applications. The devised application in the system's video analytics component is therefore implemented on top of Tensorflow. The TensorFlow Object Detection API [53] is yet another open-source project built on top of Tensorflow which provides abstract APIs for the swift and effortless deployment of object detection models. As such, this work uses the above detection API for the deployment of vision algorithms.

Vision Algorithms For the purposes of this work, a couple of state-of-the-art vision algorithms are integrated into the video analytics component to perform two sets of distinctive analytics functions, described in more detail in what follows:

- **Object Detection and Classification** MaskRCNN [54] is used for object detection and classification in the video frames. The algorithm uses a pre-trained model to detect the presence of objects of interest in a video frame. Additionally, for a predefined set of object

classes, the algorithm infers the particular class each detected object belongs to. The pre-trained model used in this work also referred to as the *frozen inference graph*, is a publicly available model trained on the COCO dataset [55] objects. Therefore, the frozen graph is obtained from the Tensorflow Detection Zoo [56] and incorporated into the video analytics component. Moreover, the employed deep neural network architecture is that of Inception ResNet-V2 [57].

- **Object Tracking** DeepSORT [58] is a vision algorithm based on deep neural networks to solve multiple object tracking problems. DeepSORT algorithm allows for reidentification of a detected object in a video frame, throughout the subsequent frames, hence providing ways to intelligently track the movements of an object of interest in a video stream. Therefore, in addition to the MaskRCNN algorithm discussed above, the DeepSORT algorithm is also integrated into the proposed systems video analytics component. The DeepSORT algorithm expects as input the coordinates of the objects that need to be re-identified and tracked. The coordinates for the detected objects of interest produced by the MaskRCNN algorithm are subsequently fed to the DeepSORT algorithm. It should be noted that for this work, the DeepSORT algorithm is used for the tracking of detected objects in one single video input stream.

4.3.6 Physical Design of Data Pre-Processing and Enrichment

Whereas the physical design of Data Pre-Processing and Enrichment is as a practical matter, likely to include several additional modules and components, the three essential comprising parts thereof are briefly discussed in what follows.

InfluxDB Writers As discussed in Section 4.3.5, a large set of metrics, statistics, and data is produced by the proposed system’s video analytics component and later published to the system’s distributed message queues.

As the reader may anticipate, a considerable portion of the latter component’s data is time-series in nature and is therefore, committed to the system’s underlying time-series data stores. The InfluxDB Writers’ task in the Data Pre-Processing and Enrichment component is to accomplish the preceding job.

As discussed earlier, the video analytics component publishes each set of data in a standard JSON format; however, the schema varies among different data types. On the other hand, each of the underlying time-series data stores expects the input data's schema to conform with specific requirements and hence, follow a coarsely defined schema.

As such, the InfluxDB Writers consume the published streams of data under a Kafka topic, performs the required data transformations on data and ultimately, commits the resulting data to the InfluxDB database.

Moreover, in addition to the data transformations, the data is enriched by adding new data fields or updating the existing ones as necessary. In the proposed system, the latter occurs when, for instance, the time taken for a message to traverse each of the system components along the way needs to be computed.

MongoDB Writers These processes perform essentially the same functions as the above InfluxDB writers storing however a different set of data as discussed Section 4.3.4, and consequently, storing it to a different distributed data store, *i.e.*, MongoDB.

Frame Writers As the name implies, the frame writers are devised to store the entire set of the retrieved video frames to the persistent storage. The frame writers, likewise the writers above, consume the frames published to the message queues and accordingly persist it under the paths and URI⁷s that follow a pre-established naming convention, such that they can be easily retrieved in future.

4.3.7 Physical Design of Data Security

As described in Section 4.2.8, the proposed system's security aspect is dealt with, in a couple of ways. First, the end-to-end encryption of the communication is made whenever the communicating parties make use of a publicly accessible network infrastructure to establish a connection. Second, the access to the running system services is restricted to the small set of participating nodes in the cluster. Finally, wherever feasible and applicable, the communication among the services is secured via SSL/TLS.

⁷Uniform Resource Indicators

Secure Communication

a) Secure Tunnels for Non-Swarm Services Encrypted tunnels are used to secure the communication among the nodes wherever necessary. The tunnelling protocol used is OpenVPN [59]. OpenVPN is open-source and provides a great level of versatility and flexibility for the selection of the particular algorithm to use to authenticate the communicating parties, perform secure key exchange or alternatively, authenticate and encrypt exchanged data packets.

The particulars of the encryption and message authentication protocols used to secure control and data channels of an OpenVPN tunnel and their respective cryptographic robustness fall outside the scope of this work. However, the rationale behind the its inclusion in the system’s physical design, its feasibility and how it has the potential to fulfill the real-time requirements of the application are briefly discussed.

An OpenVPN connection is composed of two disjoint communication channels multiplexed together, namely, *control* and *data* channels [60]. Parties at each end of the encrypted OpenVPN tunnel, first initiate a TLS session over the control channel to exchange and negotiate the cryptographic keys and ciphers needed to encrypt and authenticate the transmitted packets. Once concluded, the encrypted packets are forwarded through the data channel.

Moreover, OpenVPN tunnels can be established using either one of the UDP or TCP transport protocols. In using the UDP protocol for transport, the separation of the control and data channels in OpenVPN allows ciphering all communication in the control channel via a robust and reliable TLS session over the UDP connection. In contrast, the data channel is used to only forward encrypted packets, with absolutely no overhead other than encryption and authentication of each forwarded packet.

Hence, the above design choice should safely enable secure communication, with little to no performance penalty in meeting the application’s real-time requirements.

b) Encrypted Overlay Networks for Swarm Services As discussed in Section 4.3.1, each group of services running as a *stack* in the Swarm cluster is configured to communicate with each other via an underlying overlay network provisioned for them. The overlay network created by Docker’s network driver does not use encryption by default to reduce the incurred overhead. However, it provides the option to encrypt the overlays network traffic using AES [34]. Therefore,

to provide an additional security layer, the AES algorithm will be used to encrypt traffic for the overlay network using a symmetric cipher key.

Packet Filtering Rules

Data security can further be enhanced by restricting access to the cluster nodes and hence, blocking any incoming traffic from outside the cluster that is destined to the cluster nodes. As a practical matter, the only exception would be for the traffic destined at the Secure Shell (SSH) [61] port of each running node.

To that end, the Netfilter [62] framework provided in the Linux Kernel is used, and the corresponding packet filtering rules are accordingly created to accordingly drop any unwanted incoming traffic destined for the nodes.

4.3.8 Physical Design of Presentation

The presentation component must connect to the time-series databases containing the set of collected metrics and retrieve and visualize the collected metrics in real-time. Moreover, it should provide suitable plugins to connect to each of the time-series databases used in the proposed system's physical design, *i.e.*, OpenTDSB and InfluxDB.

To that end, the open-source visualization tool, Grafana [63], is used. Grafana has a rich set of visualization graphs and plots. It also meets the proposed system's requirements as it does well integrate with both OpenTSDB and InfluxDB, featuring configurable plugins for both the input data sources.

4.4 Summary

The proposed system's architecture discussed in the previous chapter and the defined system requirements thereof, were used to carry out the actual system's design in this chapter. The system's design was accomplished in two consecutive stages, namely, the logical and the physical design.

The system's logical design was carried out by first defining the logical data flows among the system's comprising components. Therefore, the system's logical design focused on the

comprising entities and modules of each system’s architectural component.

Moreover, where applicable, the required interfaces and standards for the system’s logical components’ interconnection were discussed and determined. The logical design also included the design alternatives and the analysis of their suitability and aptness of use in satisfying the previously established system requirements.

The system’s physical design was carried out posterior and consequent to the foregoing, thereby making a detailed analysis and discussion of the specific existing frameworks and technologies that explicitly enable the proposed system to effectively deliver on the pertinent set of system requirements.

Whereas the physical design of the video analytics component was devised from scratch to allow for the integration of the video analytics workloads employed in this work, the physical design of several other system components instead involved leveraging the existing widely-used and open-source big data technologies.

In carrying out the physical design of the preceding components, the rationale and grounds for inclusion of each technology was provided and discussed. Additionally, where applicable, the particular set of configurations needed to meet the non-functional attributes of the system were explained and discussed.

The actual deployment and evaluation of the system is carried out in the next chapter.

Chapter 5

System Evaluation and Results

5.1 Overview

In this chapter, the actual deployment of the proposed real-time video analytics system is carried out. The physical infrastructure used for the deployment of the application is described in Section 5.2. Therefore, the deployed system is evaluated by executing several experiments. Information on the conducted experiments is outlined in Section 5.4. Ultimately, the obtained results through experiments and the findings' discussion are presented in Section 5.5.

5.2 Physical Infrastructure and Testbed

This section describes the physical infrastructure and hardware resources used in order to implement the proposed system in this work. To fulfill the system's implementation requirements, the required resources were acquired from two different cloud service providers.

The first cloud infrastructure provider is that of SAVI,¹ which stands for Smart Applications on Virtual Infrastructure and is an NSERC² Strategic Network project established to enable investigation of important elements of future application [64] and of particular interest to this work, it provides the virtualized infrastructure to support experimental research in cloud computing.

¹Smart Application on Virtual Infrastructure. <https://www.savinetwork.ca/>

²Natural Sciences and Engineering Research Council of Canada. <https://www.nserc-crsng.gc.ca/>

The second cloud provider is that of SciNet,³ the Advanced Research Computing at the University of Toronto, and it provides computing infrastructure and powerful computational resources to Canadian researchers.

All the required components of the proposed system in this work are implemented using various resources acquired from both of the cloud providers, as mentioned above. The required computing resources consist of several virtual or physical machines, all with generic computing capabilities. However, at least one computing machine must also have a dedicated graphics processing unit (GPU) installed.

GPUs have a highly parallel structure compared to general-purpose CPUs and are thus much more efficient for manipulating computer graphics and image processing. Noteworthy and of significant importance to this work, the GPUs can exhibit orders of magnitude superior performance in applications involving deep learning compared to general-purpose CPUs [65, 66].

A total of five Virtual Machine (VM) instances of **xlarge** flavour were acquired from the SAVI network. The particular hardware configuration used for the **xlarge** instances on the SAVI network and details of the installed operating system and kernel versions are summarized in Table 5.1.

All of the VMs on SAVI are provisioned from a single region and reside on the same network subnet. Additionally, it should be noted that the acquired VMs have generic computing capabilities and none has a dedicated GPU installed.

Therefore, an additional server with a dedicated GPU device installed is acquired from the SciNet network and added to the above cluster used as a testbed for the proposed application. A summarized description of the server is shown in Table 5.2. The server is primarily used for the deployment of deep learning vision algorithms required by the system’s video analytics component. Network communications with the rest of the servers on the SAVI network are secured by means of an encrypted tunnel. Figure 5.1 illustrates the resulting testbed configuration from the above.

³The supercomputer centre at the University of Toronto. <https://www.scinethpc.ca/>

Table 5.1: System Configurations for the SAVI Acquired VM Instances

Description	Value
Node Type	Virtual Machine (VM)
VM Instance Type	<code>xlarge</code>
CPU Family	Intel Core (Broadwell IBRS)s
CPU Cores	8x
Memory	16 GB
Disk Space	160 GB
Linux Kernel Version	4.4.0-178-generic x86_64 (64 bit)
Docker Engine Version	19.03.08 (built v. <code>afacb8b7f0</code>)
Operating System	Ubuntu 18.04.3 LTS

Table 5.2: System Hardware and Software Configuration of SciNet’s Physical Machine

Description	Value
Node Type	Physical Machine
Server Model	Supermicro SYS-1029GP-TR (model X11DPG-SN)
CPU	Intel Xeon Gold 5218s (2x)
CPU Cores	64 Cores
Memory	192 GB
GPU	See Table 5.3
Linux Kernel Version	4.15.0-55-generic
Tensorflow Version	1.15
MaskRCNN Model Name	<code>mask_rcnn_inception_resnet_v2_atrous_coco_2018_01_28</code>
Docker Engine Version	19.03.08 (built v. <code>afacb8b7f0</code>)
Operating System	Ubuntu 18.04.3 LTS

Specific hardware details concerning the GPU device used to speed up the object detection and classification task is included in Table 5.3.

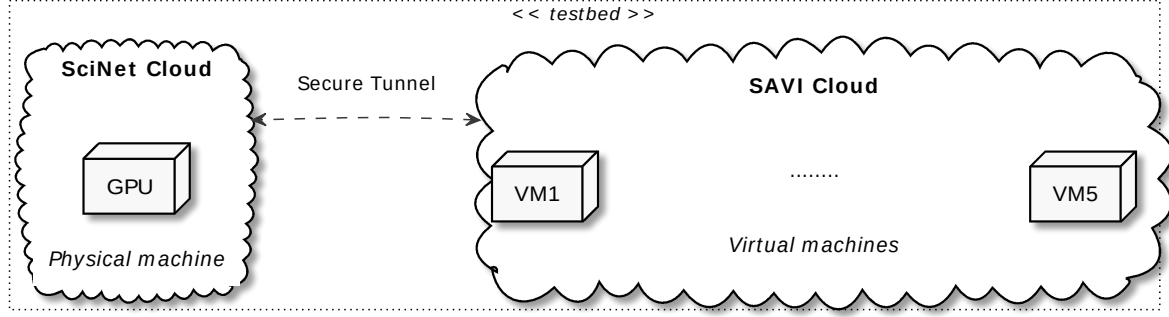


Figure 5.1: Proposed System Deployment's Physical Infrastructure and Testbed

Table 5.3: System Deployment's Dedicated Graphics Processing Unit (GPU) Specifications

Description	Value
GPU	NVIDIA GV100GL [Tesla V100 PCIe]
GPU Memory	32GB
GPU Memory Bandwidth	900 GB/sec
CUDA Version	v10.2.89
NVIDIA Driver Version	440.33.01
CUDA Capability	7
CuDNN Version	7.6.5
NVIDIA CUDA Cores	5,120
NVIDIA Tensor Cores	640
GPU Architecture	NVIDIA Volta [67]

5.3 Evaluation Methodology

The proposed system is composed of a reasonably extensive number of components and performs a considerably large number of functions internally. Consequently, a wholistic probing and inspection of the system inevitably requires an extensive set of experiments to be meticulously designed, executed and analyzed.

Therefore, the evaluation mainly focuses on the system's significant and essential aspects

instead, thereby shedding light on how well such a design can deliver on its objectives. Specifically, the evaluation section is primarily concerned with probing and analyzing the system's most significant performance indicators, when the system is viewed as a whole, including the system's throughput, resource demand, response time, and latency encountered throughout the applications' execution.

In simple terms, the proposed system aims at enabling the real-time execution of any number of video analytics jobs in a distributed and scalable fashion. Consequently, the evaluation of the system involves the inspection of several metrics, each discussed in what follows.

Response Times and Latency

Several design choices were made with due consideration of the inherent real-time requirements of video analytics applications. The real-time term is loosely defined yet widely adopted, and its specific meaning heavily relies on the particulars of the use case. The real-time requirement of some big data applications is quite stringent and typically in the range of at most a few dozens of milliseconds, as is the case of, for instance, a remote surgery application.

However, while the application's real-time requirement varies with the specifics of the use case, most likely than not, a large majority of video analytics applications, the system can safely tolerate up to a few seconds of latency without falling short of its intended objective and functionality.

The system's end-to-end time can be defined as the time taken by the system to extract a video frame from the input video feed and present the results of video analytics jobs performed on it to the end-user.

The latter end-to-end time represents the response time of the system, and it is mainly composed of components illustrated in Figure 5.2, which is described in what follows:

- **Video Analytics Job Execution Time** This includes the taken by the video analytics application in connecting to the input video source, extract a frame and, therefore, perform any particular set of required analytics jobs on the frame.
- **Publish and Update Time** This includes the time taken to publish the results of the analytics jobs to message brokers and the time required for the corresponding consumers

and content updaters to retrieve the data and accordingly update the underlying storage systems.

- **Presentation Time** The time taken by system dashboards and presentation components to perform user-defined queries on the underlying time-series data stores and visualize the obtained results. The presentation time is mainly determined by the query response time exhibited by each of the underlying time-series databases. Moreover, whereas data may be queried for any interval of time, the application’s real-time nature suggests that a predominant portion of queries would involve, at a maximum, just a few minutes worth of data points. Therefore, compared to other contributing factors, this part is unlikely to significantly contribute to the overall end-to-end delay experienced by the system.

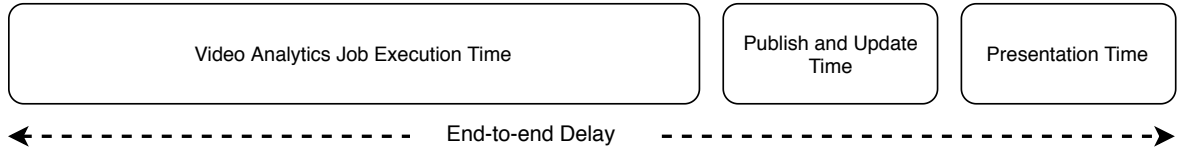


Figure 5.2: Constituting Components of the End-to-end Delay

Therefore, evaluation of the system includes measuring the time taken by each one of the above components to also additionally identify the component with the most significant contributions to the system’s response time.

Throughput

Input video sources each can have different frames captured per second and diverse image resolution for frames. The video stream produced by a conventional surveillance camera typically contains anywhere between 30 to 60 frames per second.

However, rarely a video analytics job requires processing all the input frames at such a high rate. The objects within the region monitored by a camera device are unlikely to transition at such fast rates.

Hence, the sampling rate, *i.e.*, the rate with which a frame is selected and extracted for analysis, varies depending on the system’s particular use case and could be as large as a single frame every half-a-minute.

Regardless of the particular use case, it is notably valuable to measure the system throughput, which may be expressed as the number of frames that the deployed system can process per second while still satisfying the real-time requirements.

Resource Demands

The utilization pattern of computing resources is often an indispensable prerequisite for capacity planning and performance tuning.

The monitoring component of the proposed system, thoroughly described in Chapter 4, collects an extensive set of performance metrics across all computing resources available throughout the cluster and is therefore used in to inspect the utilization pattern exhibited of a subset of resources with the most significant contributions to both system throughput and response time.

Often a system reaches its maximum throughput upon utilization of the underlying computing resource reaching its capacity. However, in practice, several factors, including inefficiencies in the executing code or insufficient or sub-optimal tuning of system parameters, often impede a system reaching its maximum throughput.

The latter is often accompanied by, and usually consequent to, an inefficacious utilization of available computing resources. As such, insofar as the proposed system’s throughput discussed in the previous section is concerned, the utilization of the GPU resource is inspected and evaluated.

The emphasis on such a resource stems from the anticipated expectation of the load the GPU is likely subjected to, given the complexity of inherent to deep learning algorithms needed for the carrying-out of required video analytics tasks.

Performance Gains in Parallel Execution of Video Analytics Jobs

This work’s devised video analytics component aims to maximize the system’s throughput by executing multiple video analytics tasks in parallel instead of sequential execution. As such, the system’s performance gains in terms of resource utilization and throughput, if any, is therefore probed and evaluated.

Overheads Incurred in Securing System Data

Encryption of traffic among participating nodes in the system deployment, while safeguarding and providing a reasonable level of protection for data against potential eavesdropping or malicious attacks, yet can negatively impact systems' end-to-end latency and will invariably result in increased utilization of both CPU and network bandwidth. As such, the actual cost incurred as a consequence is evaluated.

System's Response to Changes in Input Load and Type

Demands imposed on system computing resources are varied depending on the specific attributes of the video streams that are input to the system for analysis. In particular, the required processing time and resource utilization is varied depending on the particular resolution of video frames that are input to the system. Hence, the system is evaluated with a varying set of input video frame resolutions.

5.4 Experimental Design

5.4.1 System's Input Source

The evaluation of the system requires a video streaming source to be used as input. In practice, such a video source will most likely constitute a surveillance camera device on the customer premises or a network video recording (NVR) device therein, whereto the camera devices are directly connected.

However, for system evaluation purposes, the particular source of the video is likely of no consequence, provided however that the video feeds are served from a remote location, and the corresponding attributes and properties of the video in terms of resolution and video framerate are no different than what a typical surveillance camera device provides in practice.

As such, the system is instead evaluated by first hosting and serving the video content from a remote server. Whereas any conventional server can serve as the aforesaid video hosting server, online public video-sharing platforms such as Youtube⁴ prove as robust and reliable alternatives as they do essentially perform the same intended function.

⁴Online video-sharing platform headquartered in San Bruno, California. <https://www.youtube.com>

Therefore, for each of the conducted experiments in this section, the particular video content needed as input is henceforth uploaded to and served from the above platform.

5.4.2 Workloads

The proposed system is evaluated using several workloads to better probe the system's response to workload changes in terms of resource utilization and response time. To that end, the execution model of video analytics tasks and the image resolution of video streams are varied across experiments. What follows is a detailed description of the foregoing.

- **Different Input Video Resolution** An important factor contributing to the video analytics module's overall time to process incoming video frames is that of the image resolution of frames themselves. A higher resolution for the video frames will generally result in more accurate video analytics results. A larger number of pixels is now used to render each object, and hence, the video frame has a superior level of detail associated with it.

However, it also means that more pixels need now be processed to detect and classify any objects of interest present in the video frame. Therefore, to examine how an increase in frame resolution impacts the processing time required for the inference job, for a single video source, the application is executed multiple times, each time with a different configuration of input stream resolution.

The system is evaluated using two widely used video content resolutions, commonly referred to as Low Definition (LD) and High Definition (HD). However, it should be noted that the foregoing terms are likewise occasionally referred to as low and high-resolution inputs, respectively, and throughout the rest of the document herewith. The exact number of pixels for each type is included in Table 5.4.

- **Parallel vs. Sequential Execution of Video Analytics Jobs** As discussed in Chapter 4, the video analytics application devised as part of this work does make use of multiple object detection workers that are executed in parallel to maximize both system's throughput as well as utilization of available resources.

Therefore, further evaluation of the system is required to determine the degree to which the latter objectives can be met, employing the approach used in the devised analytical application.

As such, for each video source, the application is executed twice. In the first run, frames extracted and queued for processing are fed to the object detection and classification module sequentially, *i.e.*, the video analytics job is busy processing at most one video frame at any one time.

In contrast, the second execution involves using multiple processes running in parallel and each working on one of the several frames queued for processing.

A summary of the experiments described above is included in Table 5.4. For better readability, a shorthand name is associated to each of the above experiment that aims to indicate the particular video resolution of the input and is followed by one of the **seq** or **par** keywords to indicate whether the application is processing the queued frames sequentially or in parallel, respectively. Workloads are summarized in Table 5.4.

Table 5.4: Summarized Description of the Workloads Used in the System's Evaluation

		Execution	Number of	Input Video Resolution	Input Video
No.	Name	Model	Workers	(pixels)	Resolution
I.	LD Seq	Sequential	1	640x360	Low Definition (LD)
II.	HD Seq	Sequential	1	1280x720	High Definition (HD)
III.	LD Par	Parallel	4	640x360	Low Definition (LD)
IV.	HD Par	Parallel	4	1280x720	High Definition (HD)

5.5 Evaluation Results and Discussion

5.5.1 Video Analytics Component

Degree of Parallelism

The section discusses the reasons for using the specific number of parallel workers above and describes the process used to determine the specific quantity used. Figure 5.3 illustrates an abstract view of the process. For conciseness, only results obtained for one of the two parallel workloads wherein high-resolution frames are used as input to the system are included (See experiment IV in Table 5.4). The application is executed multiple times, each time with an increasing number of parallel workers, *i.e.*, an increased degree of parallelism.

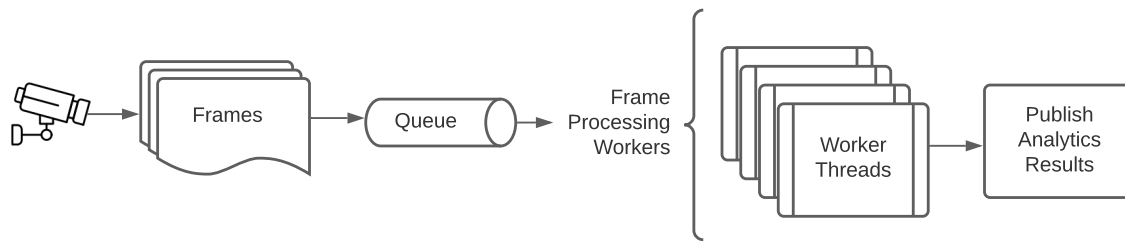


Figure 5.3: Worker Threads in the Video Analytics Component

Synchronization The devised video analytics component makes use of threads to execute video analytics jobs in parallel. However, as described in Section 4.3.5, synchronization among executing threads was added and used to prevent race conditions among worker threads.

Race conditions usually occur when data in a program’s data structure is simultaneously accessed by multiple parallel workers, resulting in failures or unpredictable behaviours for the executing threads.

Synchronization among threads is achieved by implementing locks, whereby only the single thread owning the lock is allowed access to the critical section of the program where the shared data structures reside. Consequently, the executing workers will access the program’s critical sections by taking turns and in a sequential manner.

Therefore, an overabundant use of parallel threads beyond a certain threshold will invariably result in performance degradations. The latter is primarily caused by the increasingly longer

wait times experienced by each thread in acquiring the lock.

Method In order to determine the optimum degree of parallelism to use for the tuning of the devised video analytics component, the application is executed multiple times by keeping the application load, *i.e.*, the input type and size, unchanged across executions while progressively increasing the number of parallel workers.

The above process is repeated continuously insofar as the system’s performance indicators are improving, and it is stopped when no substantial improvement is observed.

The key performance indicator measured at the end of each run is the average throughput of the devised video analytics component. The average system’s throughput is defined as the total number of input frames that can be processed in a unit of time. Therefore, the average throughput is determined by each parallel worker’s average time to execute object detection and classification algorithms on input frames.

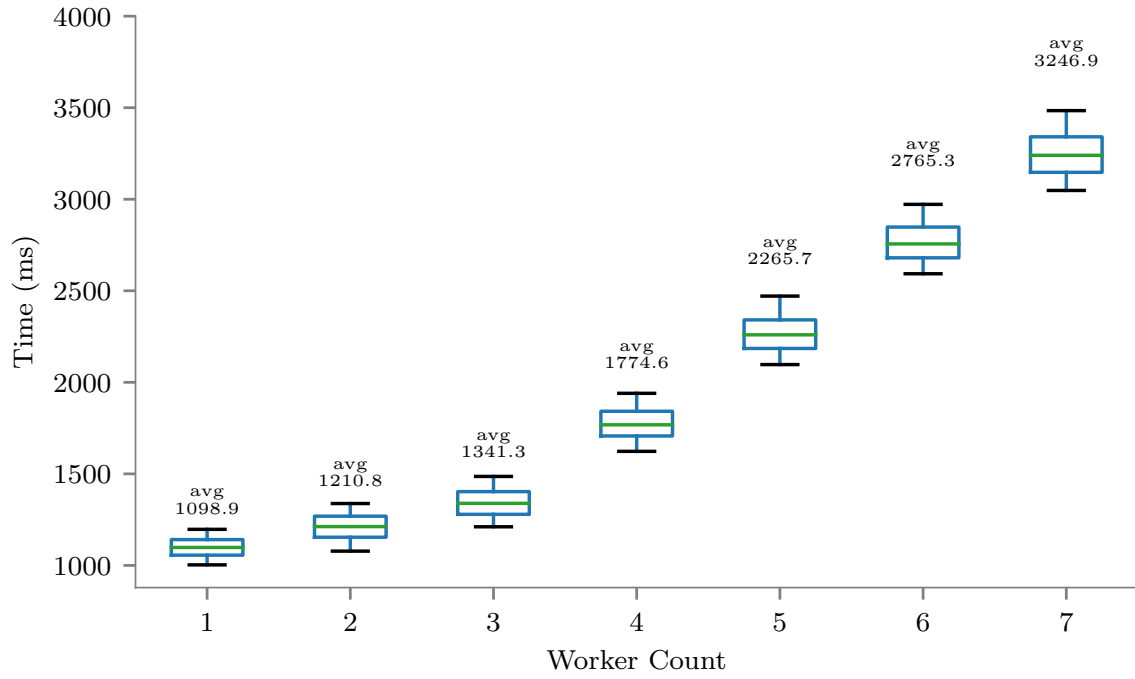
Considerations It should be noted that the hindrance in the application’s speedup caused by the required thread synchronization issue discussed above is not to be considered as the only limiting factor in achieving an increased level of parallelism.

Likewise, and on that account, the application throughput is not the only determining factor in identifying the highest degree of parallelism. However, it can serve as one, for as long as the utilization of none of the available computing resources reaches its maximum capacity.

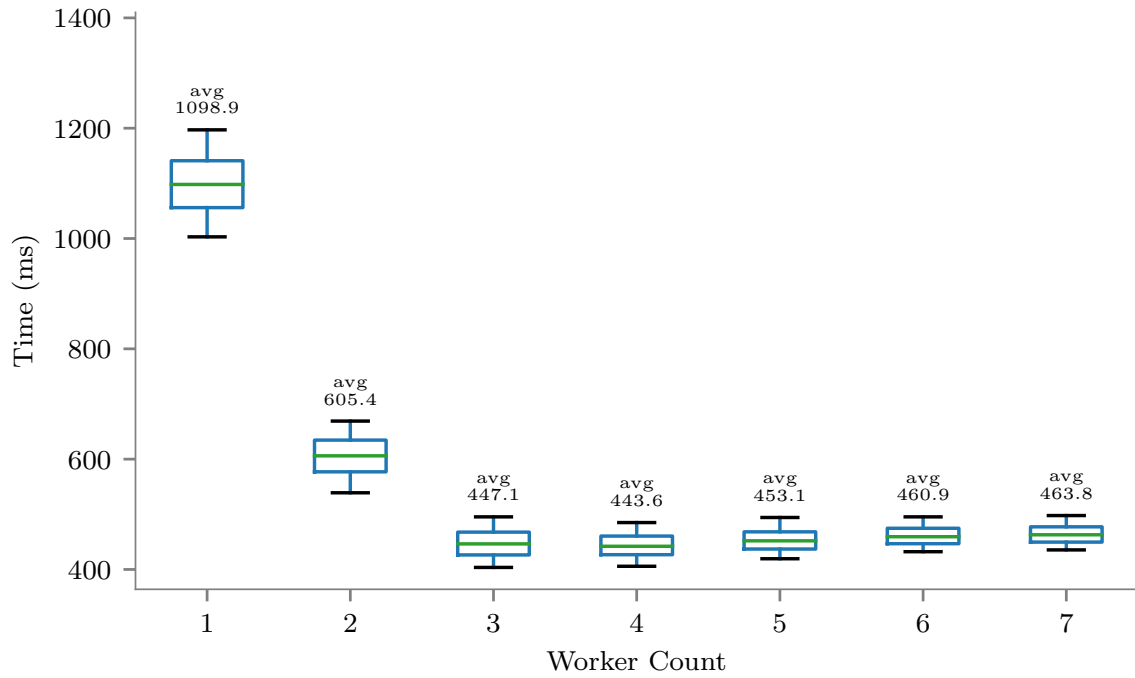
To that end, as a first step, for each configuration of parallel workers, the average time taken by each worker in performing object detection and classification is measured. As described earlier, the measurement is made possible by instrumenting the application to record the *launch time* of a worker and a corresponding *completion time* upon successful termination.

Observations Figure 5.4a illustrates the measured values for different configurations of parallel workers. Measured values show inflation in the execution time of analytics jobs resulting from an increased number of parallel workers. The inflation is due to synchronization overheads required for enabling safe parallel execution of workers.

As such, the shortest execution time manifests itself when the input frames are processed sequentially. A less pronounced yet quite significant observation can be made from Figure 5.4a



(a) Measured VA Worker Thread's Processing Time



(b) Normalized VA Worker Thread's Processing Time by the Total Number of Active VA Workers in the System

Figure 5.4: Inference Time for a Varying Number of Active Video Analytics Worker Threads

and is discussed later on in this chapter.

Figure 5.4b, on the other hand, illustrates the same preceding quantity; this time, however, each worker’s average measured processing time is normalized by the number of active workers. Intuitively, an increase in the particular degree of parallelism results in and is always accompanied by a commensurate increase in the number of video frames processed.

Quantities in Figure 5.4b can be interpreted as the system’s average time to process a frame. The lowest measured value occurs when exactly four parallel workers are active. No performance improvements are observed thereafter, and in fact, the values indicate progressive deterioration in performance as the number of concurrent workers grows.

Henceforth, for the particular deployment used in this work, whenever high-resolution frames are used as input, the degree of parallelism is set to four. The same procedure was likewise followed to determine the system’s best configuration, however, using low-resolution frames as input. Thus, values presented earlier in Table 5.4 were chosen posterior and consequent to the foregoing.

Average Processing Time per Frame

As described earlier, each of the workloads presented in Table 5.4 is executed, and the performance exhibited by the system is therefore measured and analyzed. Figure 5.5 illustrates the workflow of the process.

The particular workloads used for evaluation are selected such that the system’s response to changes in image resolution of video feeds as well as changes in the particular execution model used in performing the video analytics jobs may be analyzed.

Figure 5.6a illustrates the average time it takes the video analytics worker thread to process an input frame for each experiment. The presented values in the last two cases, where the video analytics workloads are executed in parallel, are normalized by the number of active parallel workers in the system. It must also be noted that the active parallel workers all execute on the same GPU device used for the system’s deployment. Hence, the values represent the average time taken by the system to process a single video frame.

The obtained results indicate a significant increase (approximately 1.7x) in the processing time required to perform inference on a high-resolution frame in a sequential manner, compared

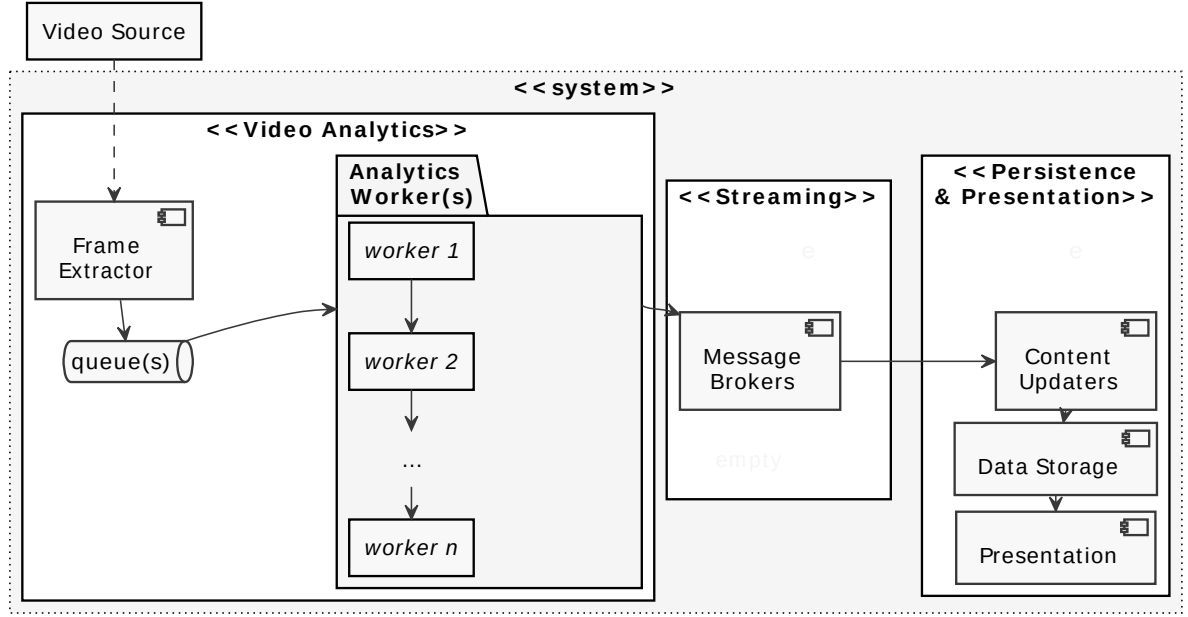


Figure 5.5: Video Analytics (VA) Component Data Flow Diagram

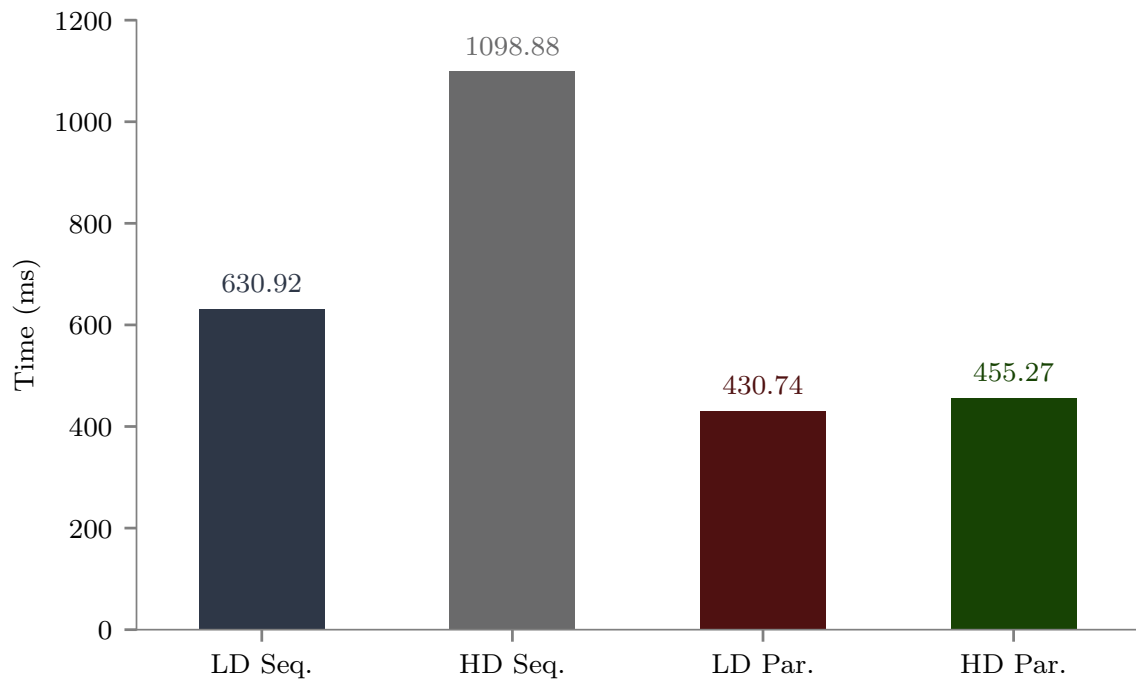
to the case where a lower resolution frame is used as input. However, the findings indicate a far less pronounced increase in the required processing time when the parallel execution model is adopted instead.

In the latter scenario, using a much higher quality of video frames as input leads to only about a 5% increase in the average processing time for each frame. Moreover, for such quality of input frames, the parallel execution is on average, resulting in an approximately 2.4x times reduction in the required processing time per each frame.

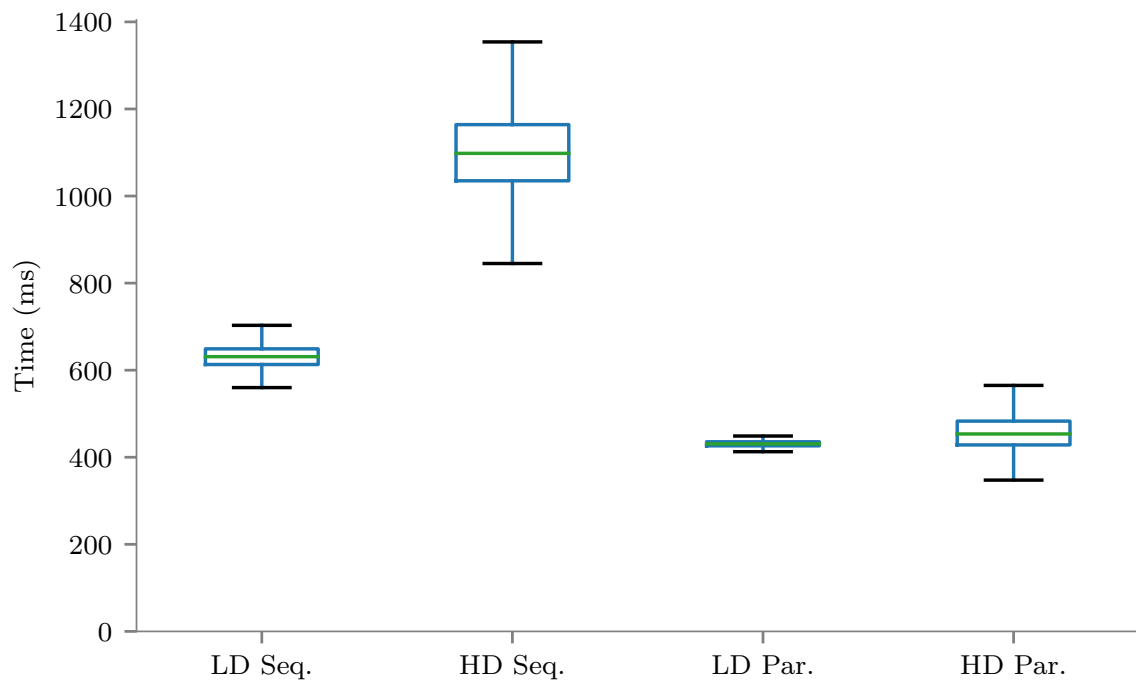
A comparatively lower performance gain is exhibited in response to changes in the application's execution model for cases where frames with lower resolutions are used as input. Intuitively, for a particular application, more massive inputs require longer processing time. The latter is consistent with the measured processing times above.

However, in a parallel execution model, the shorter completion time for a job often means that the program's critical sections will dominate a larger proportion of the entire execution time. As noted above, access to the critical section needs to be synchronized and restricted to at most one parallel thread at a time. The latter explains why the observed performance improvements are less prominent in the above case.

Figure 5.6b is a box-and-whiskers plot used to illustrate the distribution of average processing



(a) Mean



(b) Box-and-whisker plot

Figure 5.6: Deployed System's Average Processing Time Required for each Video Frame

time required for each frame for each experiment. The presented values are obtained by including all the values measured throughout the execution and, thus, dividing them by the number of active workers in each case to get the per-frame average processing time.

Throughput

System throughput is expressed as the average number of frames successfully processed and output by the system during any one-second interval. Figure 5.7 shows the overall throughput for the video analytics component for each experiment. The notation FPS is used to indicate frames (processed) per second.

The values in Figure 5.7a indicate the maximum rate with which a particular video stream may be sampled and therefore analyzed while still meeting the application’s real-time requirements. More specifically, the system can deliver video analytics results consistently at the same approximate rate.

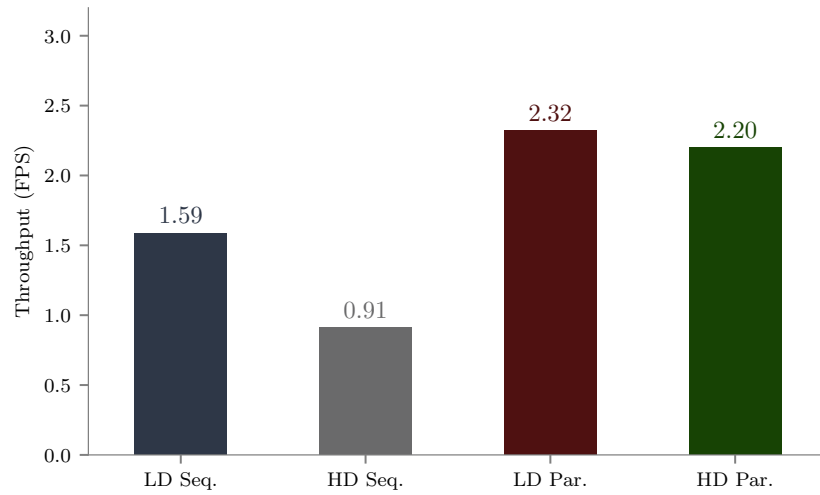
However, if the sampling rate is set above this threshold, the end-to-end delay will continually increase over time as more and more frames are piled up in the system queues for processing.

For instance, consider a particular application of the proposed system where the extraction and processing of one frame per second are required. The system set up used here for evaluation can keep up the processing at the requested rate in all cases, except for one case. The case where the input is composed of high-resolution video streams and the system is set to execute video analytics jobs sequentially.

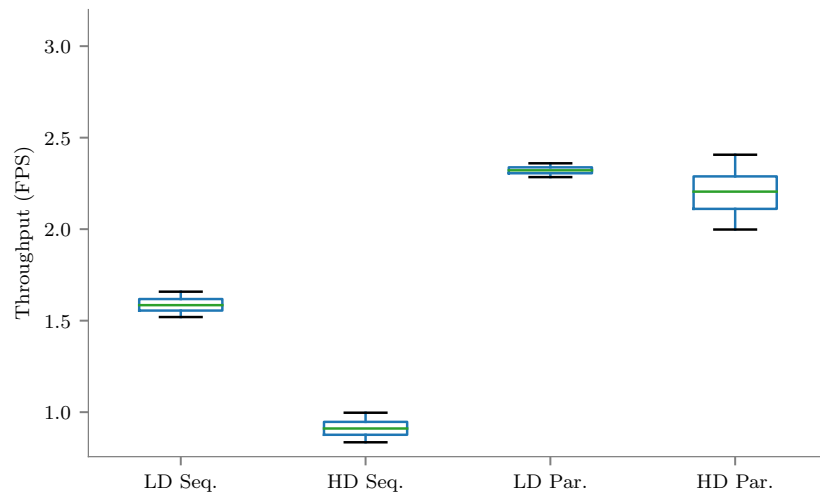
Figure 5.7b illustrates the distribution of the measured values, likewise the processing times presented in the previous section. No appreciable skewness is observed in any of the experiments, and the plot suggests that the measured values follow a normal distribution.

Data illustrated in Figures 5.7a and 5.7b indicate a higher throughput achieved by the system, with lower-resolution video inputs when the system is set to use multiple concurrent workers. Also, relatively smaller throughput variations are observed in the latter scenario compared to the case with lower-resolution inputs.

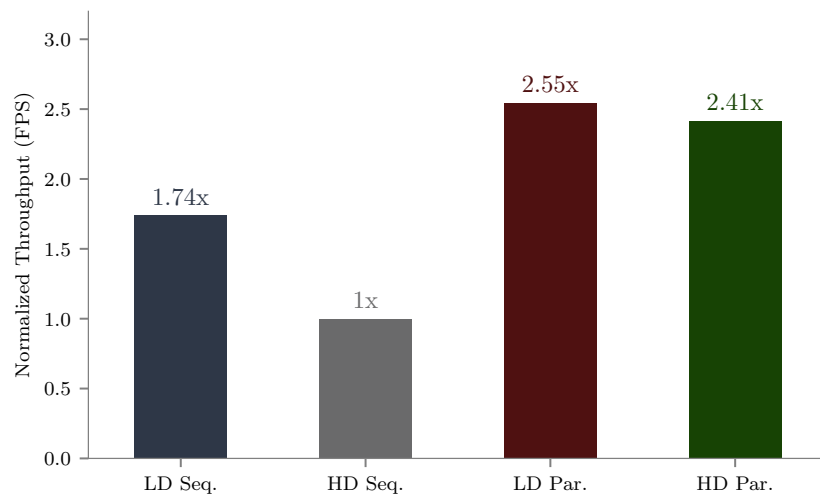
The actual values presented in Figure 5.7a are specific to the particular deployment setting used for evaluation in this work and should not be generalized. The same applies to job execution times in the previous section.



(a) Actual values



(b) Box-and-whisker plot



(c) Normalized values

Figure 5.7: Deployed System's Throughput Measurements

The exact values vary from one implementation to another, and even the same implementation is not guaranteed to yield the same values in a different deployment scenario. However, the system behaviour observed herein and discussed above is expected to manifest itself in any particular implementations and deployment scenario.

GPU Utilization

Figure 5.8 shows the resource utilization measurements made for the GPU resource in each experiment. Figure 5.8a and 5.8b show the GPU device’s utilization values and memory, respectively. Results indicate a significantly much better utilization of available GPU resources resulting from the parallel execution features implemented in the system’s proposed video analytics component. The GPU resource for both the latter cases can be observed as being utilized close to its maximum capacity.

5.5.2 Network Bandwidth Utilizations and Incurred Delays

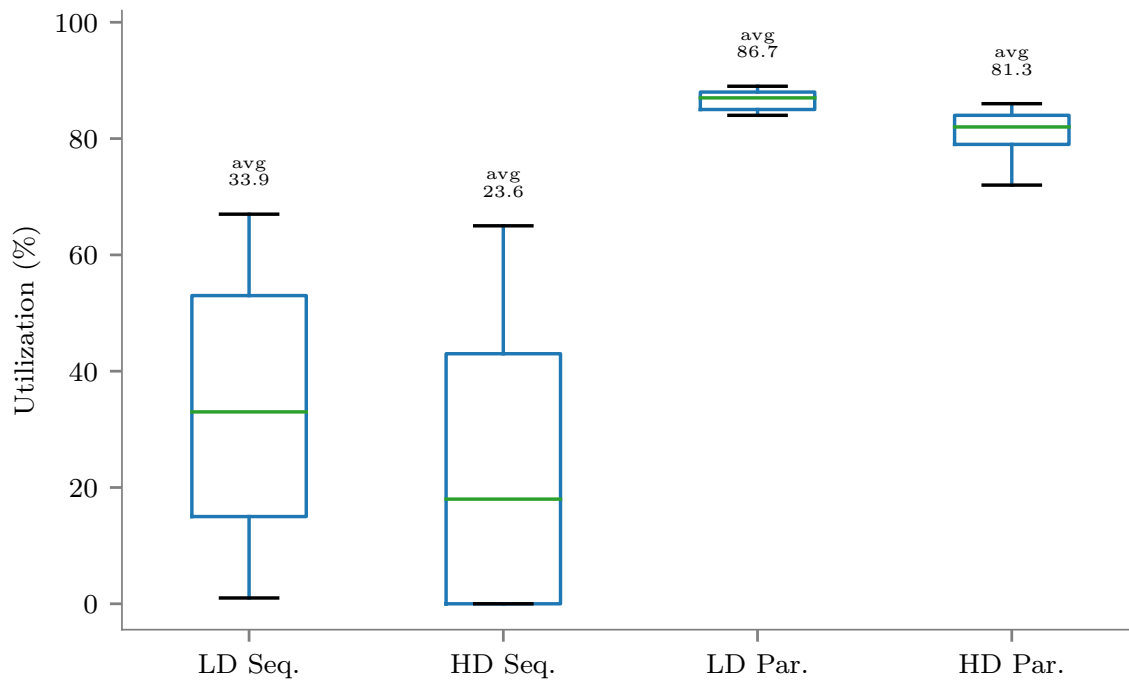
Overheads in Securing the Communication

The particular deployment configuration used in this work was illustrated earlier in Figure 5.1. As discussed, the deployment consists of resources acquired from two different cloud providers. The communication among resources residing on each provider was facilitated through an encrypted tunnel. The latter was done in part to shield data that is produced by different components of the system against unauthorized access or potential malicious cyber attacks.

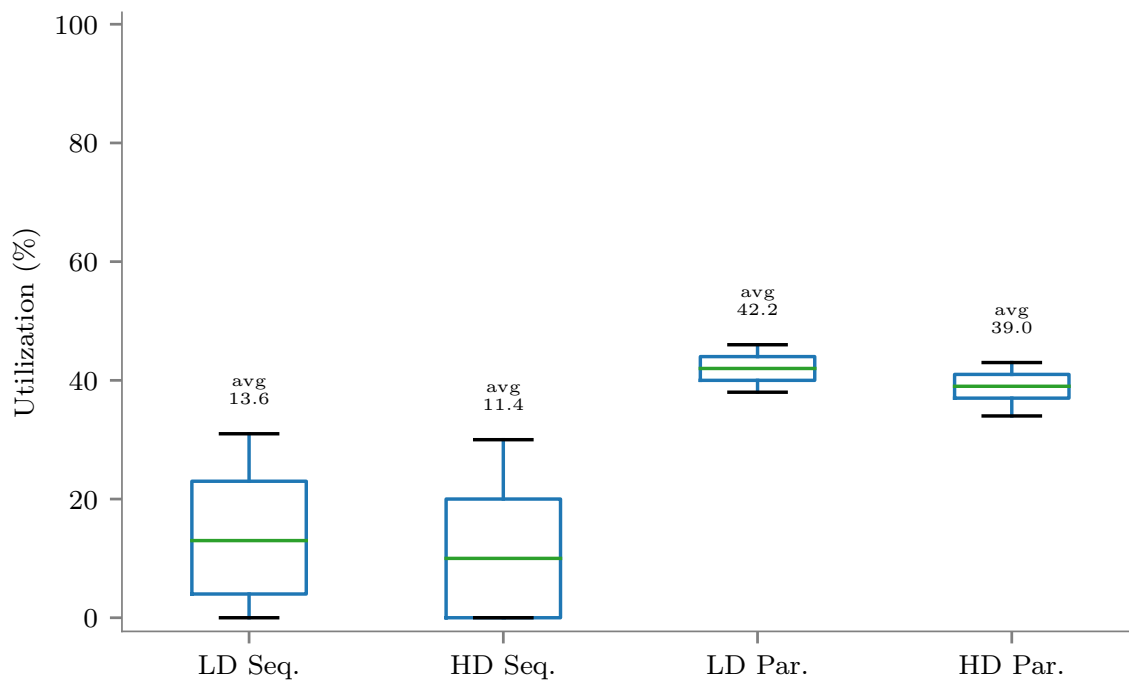
As part of the evaluation of the proposed system, it is essential to verify the suitability of choices made in Section 4.3.7. To that end, the network bandwidth utilization is measured in two cases.

First, when traffic is sent as-is, independent of whether or not any other security measures such as TLS is in place and second, when the latter traffic is forwarded through the secure tunnel discussed in Section 4.3.7. The above cases are herein referred to as insecure and secure communication, respectively.

The proposed system’s network utilization is measured under load for one hour. Figure 5.9 shows the average network bandwidth utilization between a secure and insecure communication



(a) GPU



(b) GPU Memory

Figure 5.8: Box-and-whiskers for the Deployed System's GPU Resource Utilizations

scenario. The presented values are normalized by the smallest of the two. The measured values indicate an approximately 6% increase in the network bandwidth utilization for the encrypted traffic.

In most use cases, considering the application’s actual data production rates and the advantages achieved through an increased security level, the incurred overheads are not likely to be a limitation. They can, therefore, safely get neglected.

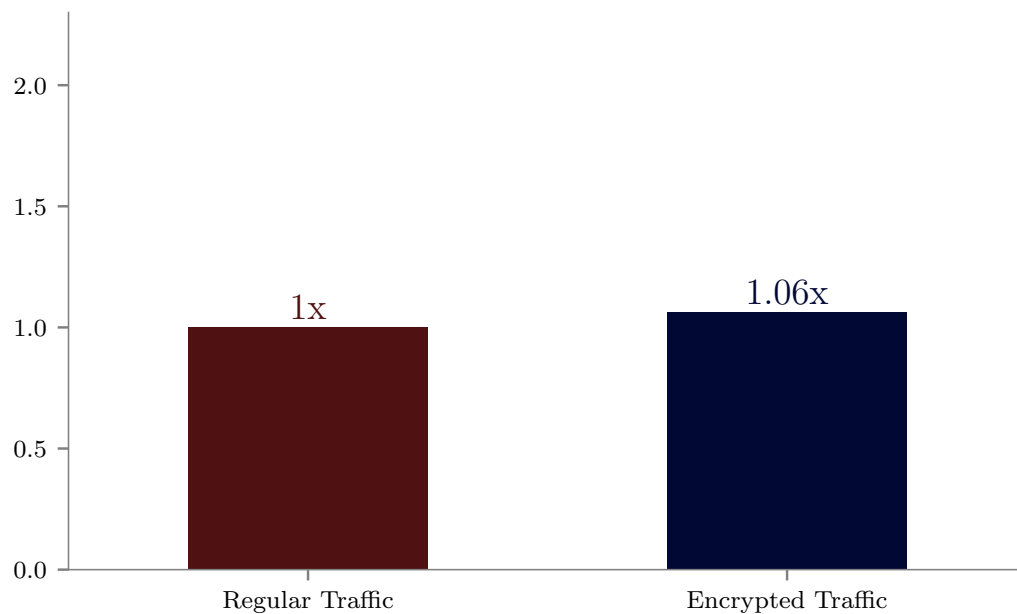
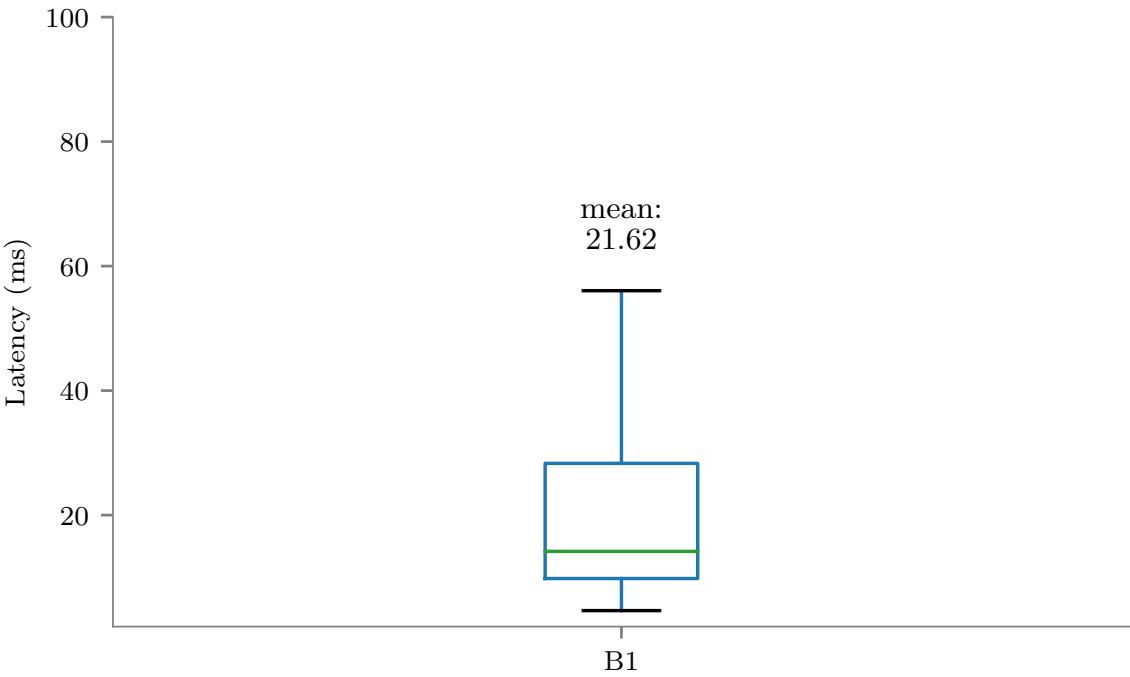


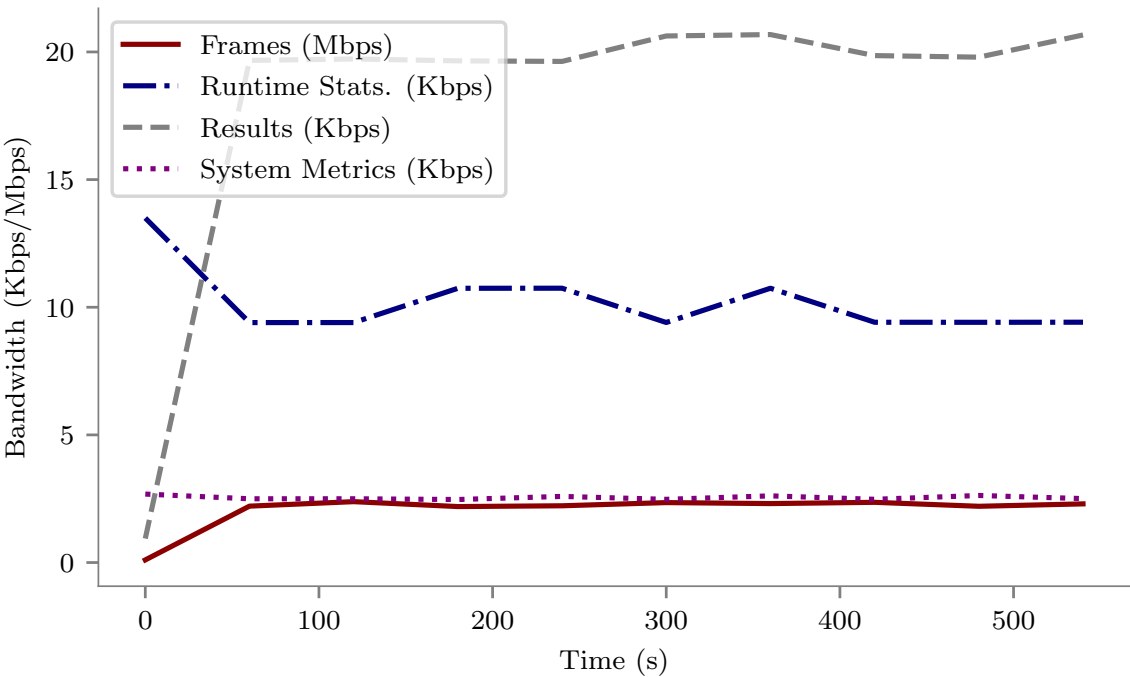
Figure 5.9: Secure vs. Insecure Communication’s Network Bandwidth Utilization

Publish and Update Time

Figure 5.10a shows the time elapsed between the moment a Kafka publisher is provided with data to publish to the broker and when the corresponding consumer receives this at the other end of the Kafka broker. The elapsed time above additionally includes the time taken by the system to write the consumer’s retrieved data to the persistent storage. Therefore, this time does not represent the broker’s actual latency; however, it includes it in part.



(a) Latency



(b) Bandwidth

Figure 5.10: System’s End-to-end Latency for Messages Sent Through the Kafka Broker and Incurred Network Bandwidth Utilizations

End-to-end Delay Constituting Components

In Section 5.3, the system’s end-to-end delay was broken down into its constituting components, and a definition for each component was provided. Table 5.5 shows a summary of the times measured for each one of the timing components contributing to the overall end-to-end delay experienced by the system throughout its execution.

Table 5.5: Time Measurements for the End-to-end Delay’s Constituting Components

Execution Mode	Average			Active VA Workers	Input Video Image Resolution
	Average Video Analytics Job Execution Time (ms)	Publish and Update Time (ms)	Average Presentation Time		
Sequential	1,098.88	20.7	$\approx$ Query time and negligible. Less than <i>1ms</i> for the most part [68].	1	HD
Parallel	1,821.09	21.62	Idem.	4	HD

Kafka Bandwidth Utilization

The tuning and tweaking of Kafka parameters to maximize throughput without introducing too much latency were described in Section 4.3.2. The implementation of the proposed system in this work uses a single partition for each one of the Kafka topics. This is done to ensure a total ordering of the events. No significant impacts on throughput were expected by the implementation of a single partition, principally due to the system’s relatively scant utilization of network bandwidth. Figure 5.10b illustrates the network bandwidth utilization incurred by each one of the topics. As observed from the values, the network bandwidth requirements for the video analytics results are significantly small. Additionally, the `linger time` parameter in the publisher was set to *5ms*. Lastly, the batching feature in the Kafka publisher is enabled and set to 16,834 bytes.

As discussed earlier in Section 4.3.5, the video analytics component’s output is used to mark

each of the detected objects in a frame with bounding boxes around them. Additionally, each of the bounding boxes is annotated with each detected object's specific class.

In this work, the proposed system's deployment publishes the above frames to the Kafka broker for persistence and visualization of data. In addition to the above, a raw copy of the extracted frame is also sent for storage. This is mainly done for testing and evaluation purposes of this work.

However, it should be noted that the last step is not essential, nor is it necessary for the correct functioning of the application as all video analytics results obtained already form part of the **results** Kafka topic in Figure 5.10b. The topic **frames** illustrated in the figure contains each of these frames.

The actual values vary for any particular deployment scenario. However, an important point to consider is the relatively low network bandwidth requirements of the application.

5.5.3 Leveraging the Video Analytics Results

MaskRCNN and DeepSORT

This section illustrates the results of the algorithms used in the video analytics component of the system. As previously discussed in Section 4.3.5, the MaskRCNN algorithm was used in the video component to perform inference and instance segmentation on the input frames.

The MaskRCNN can be used to detect objects present in a video frame and determine the particular class an object belongs to. The algorithm can be trained to detect and classify any particular set of objects. Generally, the larger the training set, the better is the quality of training for the model.

As described in Section 4.3.5, this work employs and uses pre-trained models on COCO datasets. For each class of objects, an overly extensively number of images are used to train the models. Resulting trained models can therefore be saved and reused either for the detection or transfer learning, whereby an additional set of images is used to further train an already trained model. The latter is useful, for instance, to improve the model's accuracy or tailor it to the needs of a particular scenario where the model is going to be used.

The pre-trained models used in this work are publicly available models trained to detect

objects pertaining to one of the approximately one hundred detectable classes of objects, including trucks, buses, bicycles, cars and pedestrians. As such, the trained models serve the purpose of exemplary use cases discussed in Section 3.5.6.

In addition to object detection and classification capabilities achieved by MaskRCNN, an additional algorithm, DeepSORT, was employed to perform the tracking of objects across frames as these objects move within the region of the monitored region.

Both DeepSORT and MaskRCNN algorithms use fairly complex methods and procedures to achieve each of the objectives mentioned above.

The internal of these algorithms are well outside the scope of this work. However, the particular choice of the above algorithms is driven by the significant attraction each has received from research communities in computer vision—a division in the computer science, where ways to use computer systems to infer knowledge from digital photos and images are investigated and studied.

As such, the employment of these algorithms in the video analytics component not only allows the system to derive better analytics results from the input but more importantly, it has allowed for evaluation of the system with the modern and state-of-the-art algorithms with a potential to extensively be used as part of an increasing number of video analytics applications in the near future.

Monitoring of Traffic in a Highway

As part of the system’s evaluation, a recorded video from a highway is used as input to the system. The video shows one of the frequently commuted highways in the city of Toronto and captured by the author for the purposes of evaluation of this work.

As discussed in Section 4.3.5, objects detected and classified by the first algorithm serve as input to the DeepSORT tracking algorithm. The DeepSORT algorithm aims at tracking the movements of the detected objects across any subsequent video frame. DeepSORT algorithm assigns a new and unique ID to any object for which it has no prior record and uses this ID to indicate the detection of the same object in any of the subsequent frames.

Figure 5.11 shows a sample frame output by the system. MaskRCNN algorithm produces coordinates of each object it detects through inference. The coordinates are those of the smallest

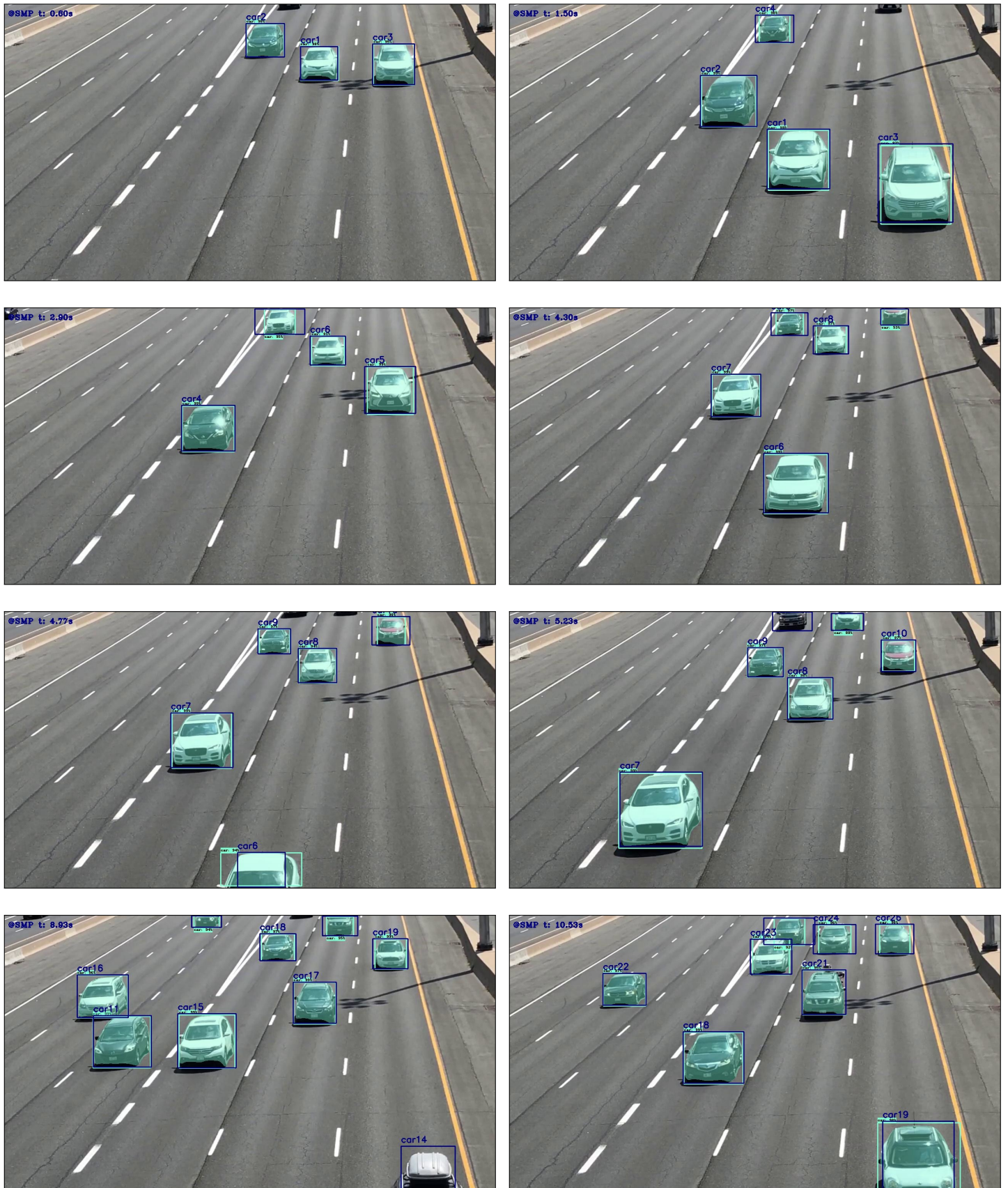


Figure 5.11: System's Detected and Tracked Car Objects in the Highway

imaginary rectangle drawn around each detected object and hence, surrounding the object.

Therefore, the above coordinates output from the MaskRCNN algorithm is used to create the green bounding boxes in Figure 5.11, representing the detected objects. Additionally, each bounding box is annotated in green colour, with the actual class the particular detected objects belong to, *e.g.*, car, truck or motorcycle.

Additionally, the MaskRCNN algorithm performs what is commonly referred to as *instance segmentation*, where the detected object's exact pixels are marked. The result is an even more superior identification of the exact location occupied by a particular object present in the video frame. The regions marked in greenish colour in Figure 5.11 represent the results obtained through instance segmentation.

Moreover, the MaskRCNN algorithm, similar to other object detection algorithms, produces a confidence value for each detected object. This value represents the algorithm's degree of confidence for the detection outputs it produces after performing inference on the frame. These values are illustrated in green, alongside each object's inferred class, in Figure 5.11. The exact values are inconsequential to the discussion being made. However, the application can be configured to require a minimum confidence level for each detection before it is accepted as a result.

The blue bounding boxes around objects in Figure 5.11 represent the outputs of the DeepSORT algorithm. The unique ID assigned by the DeepSORT algorithm is used to assign an integer count to any new object that appears in the video footage. As can be observed from the sample outputs of Figure 5.11, the commuting cars are being effectively tracked throughout the input video stream's sequence of frames. The latter can be used, for instance, to count the number of commuting cars throughout the day.

However, it must be noted that the above object tracking function in the proposed system keeps track of the movement of objects within a single video input stream. In other words, the deployed system does not make use of the information about an object identified in one video stream for re-identification of the same object as it appears in another video stream.

The results obtained through each of these algorithms can be easily aggregated with all the rest of the proposed system's metrics. Therefore, these can effortlessly get leveraged to create more complex and detailed metrics, thereby gaining deeper insights into the video frame's

information. For instance, the timestamp assigned by the application to results for each frame can be used to track the most prolonged interval a particular car remains within the region monitored by the camera.

In the scenario of a highway, the computed metric could represent the flow rate of the traffic. The persistence of a car object being tracked for a reasonably long time will indicate congestion of traffic and vice-versa. Likewise, the set of coordinates for tracked objects can be compared across subsequent frames, for instance, to compute the direction of movement for a tracked object.

The same likewise applies to other use cases. For instance, the proposed system may be likewise used to monitor a parking lot area. In that case, the detection algorithm's output can be aggregated to compute the total number of car objects in a frame. In a similar fashion, the total number of objects pertaining to a more inclusive set of classes, *e.g.*, cars, trucks and motorcycles, can be obtained.

In a parking lot scenario, the number of parking spots is known a priori and can be used in the above calculation to obtain the available parking lot capacity. Moreover, if the total number of counted cars by the application exceeds a parking lot's capacity, that would indicate the presence of congestion and could be used to trigger other actions to temporarily restrict the admission of newly arriving cars into the parking lot area.

Lastly, the DeepSORT results can likewise be used in a parking lot scenario to count the number of cars entering or leaving the parking premises.

5.5.4 Real-time Presentation and Visualization

A thorough description and discussion of the chain of components, working together to collect data across the entire system, were presented in Section 4.2.4 and 4.3.6. A snapshot of the dashboard built for the evaluation and monitoring of the proposed system is illustrated in Figure 5.12, to provide the reader with a perspective on the real-time dashboards used as part of the system. This work's use of the dashboard mentioned above is primarily to query and visualize the collected streams of data both interactively and in real-time. The presentation component was thoroughly discussed in Section 4.3.8 and as noted therein, the particular visualization tool employed is Grafana.

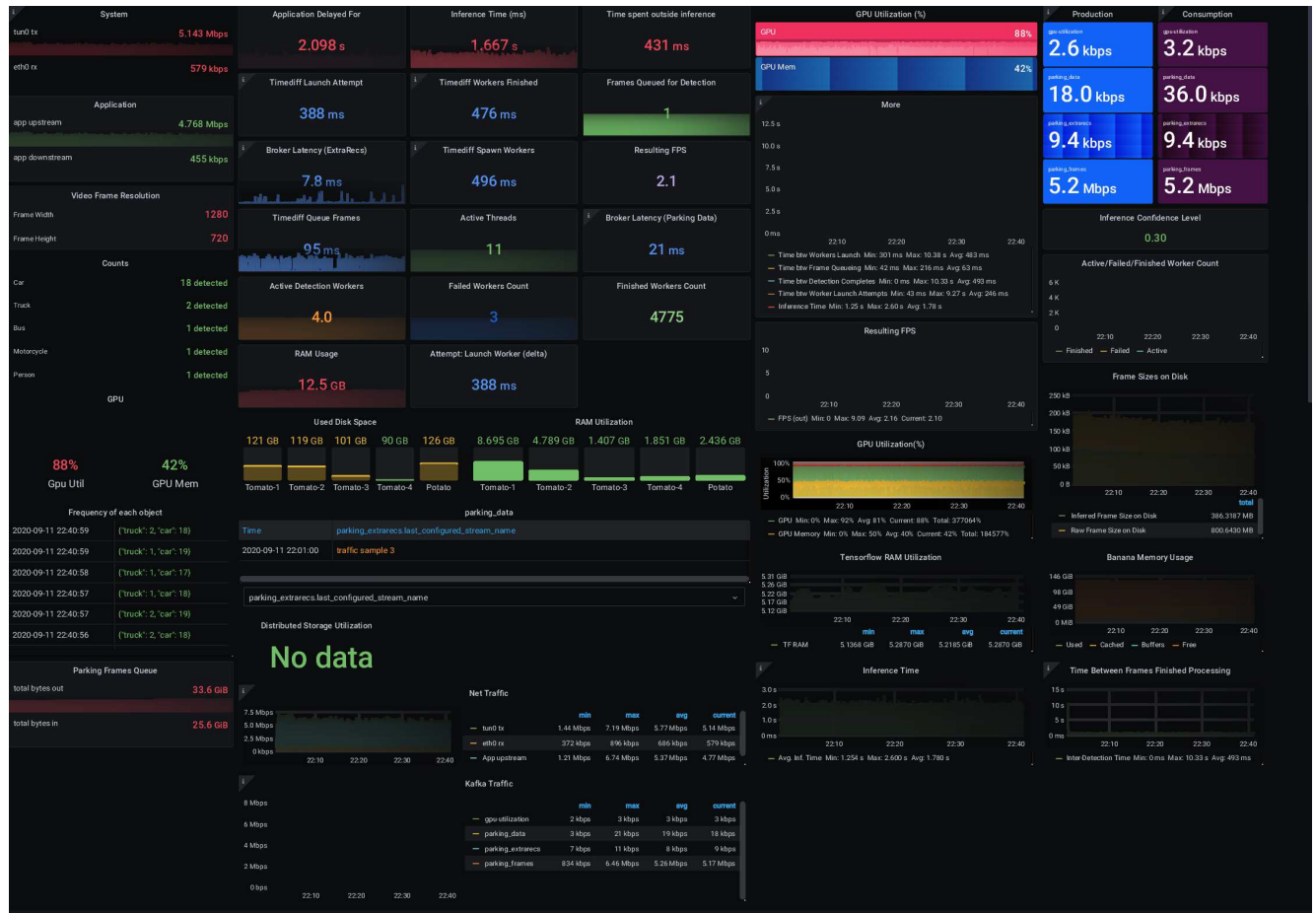


Figure 5.12: Deployed System's Dashboards used for Real-time Data Presentation

5.5.5 Implications of Parallelism on the Real-time Aspects of the System

Measured execution time for the video analytics job using a variable number of parallel workers was presented in Figure 5.6a and discussed earlier in this chapter. Table 5.6 summarizes the measured performance metrics of the proposed system, for each configuration of the number of parallel video analytics workers. It must also be noted that as discussed earlier, the parallel video analytics jobs are executing on a single GPU resource.

The obtained results demonstrate a commensurate increase in the execution time with the increase of parallel workers. As such, the shortest execution time occurs when the video analytics component is configured to use only one worker, hence, sequentially executing the analytics jobs.

Table 5.6: Measured Values for a Different Number of Active Parallel Video Analytics Workers

	Average	Increased	Mean	Mean	Average	
Active VA	Inference	Latency due to	GPU	GPU	Through-	Normalized
Worker	Time Per	Parallelism	Util.	Memory	put	System
Count	Worker (ms)	(ms)	(%)	Util. (%)	(FPS)	Throughput
1	1,098.88	-	23.58	11.37	0.91	1x
2	1,210.82	111.95	58.68	28.03	1.65	1.82x
3	1,341.25	242.37	79.93	39.08	2.24	2.46x
4	1,774.59	675.71	83.30	40.10	2.25	2.48x
5	2,265.69	1,166.81	83.67	39.87	2.21	2.43x
6	2,765.26	1,666.38	83.73	40.65	2.17	2.38x
7	3,246.93	2,148.05	83.83	40.63	2.16	2.37x

The increase in the processing time for each work stems from the fact that, in the presence of an increasing number of workers, the imposed synchronization on parallel workers described earlier above results in an increasingly larger number of workers queuing up and taking turns to enter the critical sections of the program.

However, it should be noted that in general terms, while the execution time for each worker is increased with more massive degrees of parallelism, so is the throughput of the system. Therefore, the inflation of time occurs as a natural consequence of parallelism, and for most applications, this is not an undesirable or unfavourable outcome.

However, and in particular, for the proposed system in this work, depending on the system's particular use case, the latter effect can have implications on the system's real-time requirements. In other words, while the increase in the number of parallel workers translates to higher possible sampling rates of input video feeds, it also means that the application's end-to-end latency will be consequently increased.

For instance, in this particular deployment, the values in Figure 5.6a indicate the minimum end-to-end latency the system encounters in delivering the analytics jobs results for a varying set of configurations of input resolution and degree of parallelism.

While the actual values are deployment-specific, an important conclusion from the obtained results is the trade-off between throughput and end-to-end latency required as part of the system deployment.

However, if needed, the above effect can be avoided using a variety of mitigation strategies. For instance, the input load may alternatively be spread among more than one GPU device, thereby eliminating the need for synchronization among the workers.

Nonetheless, as values in Table 5.6 indicate, it should be noted that the latter inevitably results in underutilization of each GPU resource. In any event and regardless of the particular strategy used, the above trade-off needs to be considered part of any system deployment planning.

5.5.6 Different Cloud Deployment Settings

Section 5.2 described the particular infrastructure setting used for the deployment of the proposed system. The use of two cloud service providers along with the evaluation results discussed above, provides for an opportunity to discuss some needed considerations along with the expected outcomes for several potential cloud deployment scenarios:

- A use case wherein compliance with an existing privacy law for instance, poses limitations on the transmission of captured video content to an external entity and requires the content to reside on-premises and hence, be processed locally and on-site.

The above use case scenario will closely resemble the particular deployment setup used to evaluate this work. The provider of the GPU resources, in this case, is analogous to the deployment setup that is required on the customer’s premises.

Likewise, the rest of the system required components may get acquired from a public cloud, hence leverage the elasticity, flexibility and considerably lower operation and maintenance costs offered by public cloud providers.

Such a deployment’s overall characteristics are expected to closely follow that of the particular deployment used for evaluation of this work.

- Another potential use case scenario wherein such splitting of resources across two disjoint clouds may be of interest is when the cost benefits of a typical *pay-as-you-go* service offered

by a public cloud provider are not superior to the initial savings that are attained as a consequence, in the Capital Expenditure (CAPEX) required in the initial stages of a project.

The latter scenario is more apparent when the resource in question is a costly GPU resource and required to satisfy the video analytics application's requirements.

Such cases typically involve continuous monitoring and processing of frames captured from a region of interest. Thus, in contrast to workloads that only execute ad-hoc jobs on GPU devices, the above pay-as-you-go scheme may not prove convenient.

Therefore, such use case scenario can likewise the scenario discussed above, split the resources across a private and public cloud, while still having the deployed system deliver on its design requirements.

- Edge computing is commonly considered whenever the real-time operation of an application is required. Edge computing aims at reducing the overall latency experienced by a real-time application by moving most of the processing as close as possible to where data resides.

While the concept of edge computing is generally used to refer to cloud resources that actually reside within the boundaries of a cloud, however occasionally, the same term is interchangeably used in the literature to refer to resources that are, in fact, provisioned on customers' premises. For instance, some of the existing literature refers to performing edge computing by using computing resources within the boundaries of and administered by an organization [17, 69] and hence, refer to it as cloudlets.

However, regardless of the particular definition used, what follows is expected to likewise apply to both scenarios.

Whereas for a large number of applications, the above likely constitutes the only feasible way to satisfy the application's real-time requirements; however, the findings in this work suggest this may not necessarily be the case for all real-time video analytics application.

Throughout the evaluation of the proposed system, the observations made indicated that a significantly major portion of the end-to-end latency is dominated by the time required to actually perform the video analytics jobs on the input video frames.

In contrast, the overall time taken by all the remainder system components was observed as making comparatively little contributions to the end-to-end delay.

While real-time requirements vary among deployments and use cases, the actual time required for performing analytics on video frames should therefore be a major consideration in any deployment planning.

5.6 Summary

The proposed video analytics system was deployed and evaluated in this chapter. The deployment of the entire system was carried out using computing resources acquired from two distinct cloud providers. The acquired resources consisted of several virtual machines and a powerful physical machine with a dedicated GPU device installed. Additionally, the communications between the system components deployed on each cloud were facilitated through a secure and encrypted tunnel.

To evaluate the system, several experiments were designed and conducted to probe the system's functionality and behaviour in response to system configuration and input changes. To that end, various sets of image resolution for the input video streams and different configurations for the degree of parallelism in the devised video analytics component were employed.

Therefore, measurements were made to obtain the system's average response time and throughput. Moreover, the parallel execution feature implemented as part of the proposed system's video analytics component, was probed and evaluated. To that end, the system's response in terms of latency, throughput and resource utilization with an increased level of parallelism was examined. The parameters of the devised Video Analytics (VA) component were tuned to maximize both the system's throughput and resource utilization. The methodology used for the above tuning of parameters did progressively increase the number of parallel video analytics workers for as long as the increase resulted in an improved overall system's throughput.

The obtained results show that real-time video analytics is possible as a result of the following measurements:

- The VA component was configured to execute the VA workloads sequentially. The average required time to carry out the employed VA workloads on the GPU, was $1098.88ms$ and

630.92 ms when HD and LD video images were used as input, respectively. The above resulted in a system throughput of 0.91 and 1.59 processed frames per second (FPS) for HD and LD input video resolutions, respectively.

- The system's overall end-to-end delay was broken down into its constituting components. The end-to-end delay consisted of *a)* the VA job execution time measured above, *b)* the publish and update time, which was measured as taking on average 20.7 ms and 21.62 ms for the sequential and parallel execution of the VA jobs, respectively, and *c)* the presentation time, which is approximately the same as the underlying time-series databases' query time and thus, negligible. The InfluxDB query time for the query window is at most 1 ms [68].
- The system's devised VA component was therefore configured and tuned to efficiently execute the VA jobs in parallel. The system was tuned to use 4 active parallel VA workers, for both cases with HD and LD video inputs. The increase in the number of parallel VA workers was observed to cause a commensurate increase in each of the VA parallel workers' execution times. The latter increase resulted from the required synchronization implemented for the parallel VA workers. The reader must note that this only applies to the case where the parallel VA workers share and use the same underlying GPU resource.
- The measured processing times taken by the VA worker threads executing in parallel were 1821.08 ms and 1722.96 ms , for HD and LD system inputs, respectively. Likewise, the system throughput measurements were 2.20 and 2.32 FPS, respectively. The latter indicated a 2.41 times and a 46% increase in the overall system's throughput due to the parallel execution of the VA workloads, for HD and LD inputs, respectively. Moreover, the GPU resource utilization was observed as being significantly optimized, reaching an average of 81.3% and 86.7%, for both the HD and LD video inputs, respectively.
- The parallel and thus, not sequential execution of the VA workers for LD inputs, increased the end-to-end delay for 1092.04 ms . However, on the positive side, a 46% improvement of the system throughput and an increased average utilization of the GPU resource was observed (86.7% as opposed to 33.9%). Likewise, the parallel execution of the VA workers on the HD video inputs resulted in improvements of the throughput by 2.41 times and consequently, a more efficient utilization of the GPU (from 23.6% to 81.3%). However, it also added 722.2 ms to the end-to-end delay. Nonetheless, the results presented in

Table 5.6, indicated that for HD video inputs in the system, the use of 3 parallel workers instead of 4, while decreasing the overall system's throughput and GPU utilization by 1%, causes an overhead of only $242.37ms$ to the end-to-end delay. Therefore, with such a configuration, the parallelism's impact on the end-to-end delay is greatly minimized, while still maximizing the system's throughput by 2.48 times. Moreover, as discussed earlier in this chapter, the VA component's latency can be minimized by choosing not to share the same GPU resource among the parallel VA workers. As such, making use of multiple GPUs can help to avoid the $242.37ms$ overhead mentioned above. However, the latter will inevitably also result in a reduced system's throughput and GPU resource utilization.

Chapter 6

Conclusions and Future Work

6.1 Conclusions

This work consisted of designing, deploying, and evaluating a distributed, scalable and real-time video analytics system. This was accomplished by first deriving the functional and non-functional requirements of a typical video analytics application using the set of most commonly used requirements of users/businesses in modern video analytics applications. Therefore, complete system design with due consideration of the foregoing was devised.

First, the architectural requirements of the system were devised. The architectural requirements necessitated the need for a large number of system components. The components included a video analytics engine, real-time data streaming and message queues, a variety of distributed and scalable data, visual dashboards for real-time and interactive data exploration and visualization, automated service orchestration and management as well as system-wide performance monitoring and alerting systems.

Therefore, the system's actual design was accomplished by carrying out both the logical and physical designs of the system. Hence, the entire system design was carried out by meticulous choices made for each of the proposed system's constituting components. Accordingly, the advantages and disadvantages of design alternatives were examined and discussed.

Moreover, as part of the design, a video analytics component capable of executing in parallel both object detection and classification as well as object tracking algorithms were devised and implemented.

The devised component made use of multiple threads working concurrently on performing analytics jobs on input video streams. The safe execution of the several parallel threads was ensured by incorporating appropriate synchronization among workers in the proposed system’s video analytics component.

Therefore, the proposed system was deployed in the cloud and ultimately evaluated using a wide range of different workloads. The workloads included the streaming of a prerecorded traffic video of a highway in real-time. The video analytics used the MaskRCNN algorithm to detect the car objects thereof and, thus, used the DeepSORT algorithm to track detected cars as they moved in the recorded footage.

The evaluation of the system yielded several findings, briefly discussed in what follows.

First, it was shown how both the parallel execution, as well as appropriate tuning of the devised video analytics component, resulted in significant gains in terms of both system throughput and resource utilization. Results indicated a 2.4x increase when high definition video inputs were used as input. Likewise, a 46% improvement was observed when the system input consisted of low-resolution inputs. However, any potential attainable improvements beyond that point were hindered by the underlying GPU resource, in fact, reaching its maximum utilization.

Second, it was observed that maximizing the throughput by an increased degree of parallelism invariably results in an increase in the minimum end-to-end delay or lag experienced by the system in processing and returning video analytics results.

Additionally, the impact on the network bandwidth utilization resulting from the use of encrypted communication channels in creating connectivity across participating nodes was examined. The observations indicated a negligible increase in the network bandwidth requirements.

Moreover, there was no appreciable increase in latency observed. This was due to the fact that the entire time taken throughout the portion that involved communications made through the secure tunnel was not observed to exceed a hundred milliseconds.

Ultimately, posterior to the decomposition of the end-to-end delay into its constituting components, the evaluation results indicated the overall end-to-end delay to be largely dominated by the actual time required to carry out the analytics job on a video frame. Consequently, for such circumstances, the extent to which edge computing may yield any significant improvements to the end-to-end latency was therefore called into question.

6.2 Future Work

The current work can be extended in several ways, some of which is outlined in what follows:

- Extend the proposed system to make use of multiple GPUs in the system's video analytics component
 - Improve the proposed system by employing intelligent and automated scaling of the system's required services based upon the system's input load
 - Incorporation of additional learning and vision algorithms tailored to specific application domains
 - Tailoring the proposed system to meet the specific requirements of a more specific use case
-

Bibliography

- [1] He Liu, Amy R Reibman, and Jacquelyn P Boerman. A cow structural model for video analytics of cow health. March 2020.
- [2] Deepak Vasisht, Zerina Kapetanovic, Jongho Won, Xinxin Jin, Ranveer Chandra, Sudipta Sinha, Ashish Kapoor, Madhusudhan Sudarshan, and Sean Stratman. Farmbeats: An iot platform for data-driven agriculture. In *14th {USENIX} Symposium on Networked Systems Design and Implementation ({NSDI} 17)*, pages 515–529, 2017.
- [3] Wen Shao, Rei Kawakami, Ryota Yoshihashi, Shaodi You, Hidemichi Kawase, and Takeshi Naemura. Cattle detection and counting in UAV images based on convolutional neural networks. *null*, 41(1):31–52, January 2020.
- [4] Kitae Kim, Slobodan Gutesa, Branislav Dimitrijevic, Joyoung Lee, Lazar Spasovic, Wasif Mirza, and Jeevanjot Singh. Performance evaluation of video analytics for traffic incident detection and vehicle counts collection. In *Recent Advances in Intelligent Image Search and Video Retrieval*, pages 213–231. Springer International Publishing, 2017.
- [5] Ahmad Arinaldi, Jaka Arya Pradana, and Arlan Arventa Gurusinga. Detection and classification of vehicles for traffic video analytics. *Procedia Comput. Sci.*, 144:259–268, January 2018.
- [6] Yiwen Wan, Yan Huang, and Bill Buckles. Camera calibration and vehicle tracking: Highway traffic video analytics. *Transp. Res. Part C: Emerg. Technol.*, 44:202–213, July 2014.

- [7] X Sevillano, E Màrmol, and V Fernandez-Arguedas. Towards smart traffic management systems: Vacant on-street parking spot detection based on video analytics. In *17th International Conference on Information Fusion (FUSION)*, pages 1–8. ieeexplore.ieee.org, July 2014.
- [8] Elena Màrmol and Xavier Sevillano. QuickSpot: a video analytics solution for on-street vacant parking spot detection. *Multimed. Tools Appl.*, 75(24):17711–17743, 2016.
- [9] Ming-Ching Chang, Yi Wei, Nenghui Song, and Siwei Lyu. Video analytics in smart transportation for the AIC’18 challenge. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops*, pages 61–68. openaccess.thecvf.com, 2018.
- [10] Abida Sharif, Jianping Li, Mudassir Khalil, Rajesh Kumar, Muhammad Irfan Sharif, and Atiqa Sharif. Internet of things—smart traffic management system for smart cities using big data analytics. In *2017 14th International Computer Conference on Wavelet Active Media Technology and Information Processing (ICCWAMTIP)*, pages 281–284. ieeexplore.ieee.org, 2017.
- [11] K H Cheong, S Poeschmann, J W Lai, J M Koh, U R Acharya, S C M Yu, and K J W Tang. Practical automated video analytics for crowd monitoring and counting. *IEEE Access*, 7:183252–183261, 2019.
- [12] Q Zhang, H Sun, X Wu, and H Zhong. Edge video analytics for public safety: A review. *Proc. IEEE*, 107(8):1675–1696, August 2019.
- [13] Andres Musalem, Marcelo Olivares, and Ariel Schilkrut. Retail in high definition: Monitoring customer assistance through video analytics. *MESOM*, March 2020.
- [14] R Tan, I Atmosukarto, and W H Lim. Video analytics for indoor crowd estimation. In *2018 IEEE International Conference on Service Operations and Logistics, and Informatics (SOLI)*, pages 21–25. ieeexplore.ieee.org, July 2018.

- [15] Zhuang Miao, Shan Zou, Yang Li, Xiancai Zhang, Jiabao Wang, and Ming He. Intelligent video surveillance system based on moving object detection and tracking. *DEStech Transactions on Engineering and Technology Research*, 0(iect), 2016.
- [16] Md Azher Uddin, Aftab Alam, Nguyen Anh Tu, Md Siyamul Islam, and Young-Koo Lee. SIAT: A distributed video analytics framework for intelligent video surveillance. *Symmetry*, 11(7):911, July 2019.
- [17] T Stone, N Stone, P Jain, Y Jiang, K Kim, and S Nelakuditi. Towards scalable video analytics at the edge. In *2019 16th Annual IEEE International Conference on Sensing, Communication, and Networking (SECON)*, pages 1–9. ieeexplore.ieee.org, June 2019.
- [18] Johan Barthélemy, Nicolas Verstaëvel, Hugh Forehead, and Pascal Perez. Edge-Computing video analytics for Real-Time traffic monitoring in a smart city. *Sensors*, 19(9), May 2019.
- [19] Junjue Wang, Brandon Amos, Anupam Das, Padmanabhan Pillai, Norman Sadeh, and Mahadev Satyanarayanan. Enabling live video analytics with a scalable and Privacy-Aware framework. *ACM Trans. Multimedia Comput. Commun. Appl.*, 14(3s):1–24, June 2018.
- [20] Peng Liu, Bozhao Qi, and Suman Banerjee. EdgeEye: An edge service framework for real-time intelligent video analytics. In *Proceedings of the 1st International Workshop on Edge Systems, Analytics and Networking*, EdgeSys’18, pages 1–6, New York, NY, USA, June 2018. Association for Computing Machinery.
- [21] C Bailas, M Marsden, D Zhang, N E O’Connor, and S Little. Performance of video processing at the edge for crowd-monitoring applications. In *2018 IEEE 4th World Forum on Internet of Things (WF-IoT)*, pages 482–487. ieeexplore.ieee.org, February 2018.
- [22] Junchen Jiang, Ganesh Ananthanarayanan, Peter Bodik, Siddhartha Sen, and Ion Stoica. Chameleon: scalable adaptation of video analytics. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication, SIGCOMM ’18*, pages 253–266, New York, NY, USA, August 2018. Association for Computing Machinery.
- [23] Haoyu Zhang, Ganesh Ananthanarayanan, Peter Bodik, Matthai Philipose, Paramvir Bahl, and Michael J. Freedman. Live video analytics at scale with approximation and delay-

- tolerance. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, pages 377–392, Boston, MA, March 2017. USENIX Association.
- [24] Shanhe Yi, Zijiang Hao, Qingyang Zhang, Quan Zhang, Weisong Shi, and Qun Li. Lavea: Latency-aware video analytics on edge computing platform. In *Proceedings of the Second ACM/IEEE Symposium on Edge Computing, SEC '17*, New York, NY, USA, 2017. Association for Computing Machinery.
- [25] Junjue Wang, Brandon Amos, Anupam Das, Padmanabhan Pillai, Norman Sadeh, and Mahadev Satyanarayanan. A scalable and Privacy-Aware IoT service for live video analytics. In *Proceedings of the 8th ACM on Multimedia Systems Conference, MMSys'17*, pages 38–49, New York, NY, USA, June 2017. Association for Computing Machinery.
- [26] Ashiq Anjum, Tariq Abdullah, Muhammad Tariq, Yusuf Baltaci, and Nick Antonopoulos. Video stream analysis in clouds: An object detection and classification framework for high performance video analytics. *IEEE Transactions on Cloud Computing*, 2016.
- [27] Vaithilingam Anantha Natarajan, Subbaiyan Jothilakshmi, and Venkat N Gudivada. Scalable traffic video analytics using hadoop MapReduce. *ALLDATA 2015*, page 18, 2015.
- [28] T Abdullah, A Anjum, M F Tariq, Y Baltaci, and N Antonopoulos. Traffic monitoring using video analytics in clouds. In *2014 IEEE/ACM 7th International Conference on Utility and Cloud Computing*, pages 39–48. ieeexplore.ieee.org, December 2014.
- [29] Domenico Talia. Clouds for scalable big data analytics. *Computer*, (5):98–101, 2013.
- [30] Aftab Alam and Young-Koo Lee. TORNADO: Intermediate results orchestration based Service-Oriented data curation framework for intelligent video big data analytics in the cloud. *Sensors*, 20(12), June 2020.
- [31] Shanhe Yi, Zijiang Hao, Qingyang Zhang, Quan Zhang, Weisong Shi, and Qun Li. Lavea: Latency-aware video analytics on edge computing platform. In *Proceedings of the Second ACM/IEEE Symposium on Edge Computing, SEC '17*, New York, NY, USA, 2017. Association for Computing Machinery.

- [32] G. Klyne and C. Newman. RFC3339: Date and Time on the Internet: Timestamps, 2002.
- [33] Docker. The Industry-Leading Container Runtime. <https://www.docker.com/products/container-runtime>. Online. 2020.
- [34] Joan Daemen and Vincent Rijmen. Aes proposal: Rijndael. 1999.
- [35] Apache Kafka. <https://kafka.apache.org>. Online. 2020.
- [36] Jay Kreps, Neha Narkhede, Jun Rao, and Others. Kafka: A distributed messaging system for log processing. In *Proceedings of the NetDB*, volume 11, pages 1–7, 2011.
- [37] Apache ZooKeeper: A Centralized Service for Highly Reliable Distributed Coordination. <https://zookeeper.apache.org/>. Online. 2020.
- [38] Shengan Zheng, Morteza Hoseinzadeh, and Steven Swanson. Ziggurat: a tiered file system for non-volatile main memories and disks. In *17th {USENIX} Conference on File and Storage Technologies ({FAST} 19)*, pages 207–219. usenix.org, 2019.
- [39] Philippe Dobbelaere and Kyumars Sheykh Esmaili. Kafka versus RabbitMQ: A comparative study of two industry reference publish/subscribe implementations: Industry paper. In *Proceedings of the 11th ACM International Conference on Distributed and Event-based Systems*, DEBS '17, pages 227–238, New York, NY, USA, June 2017. Association for Computing Machinery.
- [40] Netdata. Distributed, Real-time Performance and Health Monitoring for Systems and Applications. <https://github.com/netdata/netdata>. Online. 2020.
- [41] Prometheus: Monitoring System and Time Series Database. <https://github.com/prometheus/prometheus>. Online. 2020.
- [42] Telegraf: Plugin-driven Server Agent for Collecting & Reporting Metrics. <https://github.com/influxdata/telegraf>. Online. 2020.
- [43] JMX Exporter: a collector that can configurably scrape and expose mBeans of a JMX target. https://github.com/prometheus/jmx_exporter. Online. 2020.

- [44] C. Lonvick. RFC3164: The BSD Syslog Protocol, 2001.
- [45] R. Gerhards. The Syslog Protocol. Technical report, March 2009.
- [46] Rsyslog: The rocket-fast Syslog Server. <https://www.rsyslog.com/>. Online. 2020.
- [47] MongoDB: A Cross-platform Document-oriented Database Program. . <https://www.mongodb.com>. Online. 2020.
- [48] InfluxDB: Open-source Time Series Database Purpose-built by InfluxData for Monitoring Metrics and Events. <https://www.influxdata.com>. Online. 2020.
- [49] OpenTSDB: A Distributed, Scalable Time Series Database (TSDB). <https://github.com/OpenTSDB/opentsdb>. Online. 2020.
- [50] GlusterFS: A free and Open-source Software Scalable Network Filesystem. <https://www.gluster.org/>. Online. 2020.
- [51] Open Source Computer Vision Library: An open source computer vision and machine learning software library. <https://opencv.org/>. Online. 2020.
- [52] Tensorflow: An End-to-end Open Source Machine Learning Platform. . <https://www.tensorflow.org/>. Online. 2020.
- [53] J. Huang, V. Rathod, C. Sun, M. Zhu, A. Korattikara, A. Fathi, I. Fischer, Z. Wojna, Y. Song, S. Guadarrama, and K. Murphy. Speed/accuracy trade-offs for modern convolutional object detectors. In *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3296–3297, 2017.
- [54] K. He, G. Gkioxari, P. Dollár, and R. Girshick. Mask r-cnn. In *2017 IEEE International Conference on Computer Vision (ICCV)*, pages 2980–2988, 2017.
- [55] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. Microsoft coco: Common objects in context. In David Fleet, Tomas Pajdla, Bernt Schiele, and Tinne Tuytelaars, editors, *Computer Vision – ECCV 2014*, pages 740–755, Cham, 2014. Springer International Publishing.

- [56] Tensorflow Detection Model Zoo. https://github.com/tensorflow/models/blob/master/research/object_detection/g3doc/tfl_detection_zoo.md. Online. 2020.
- [57] Christian Szegedy, Sergey Ioffe, Vincent Vanhoucke, and Alex Alemi. Inception-v4, inception-resnet and the impact of residual connections on learning. *arXiv preprint arXiv:1602.07261*, 2016.
- [58] N. Wojke, A. Bewley, and D. Paulus. Simple online and realtime tracking with a deep association metric. In *2017 IEEE International Conference on Image Processing (ICIP)*, pages 3645–3649, 2017.
- [59] OpenVPN. A Secure tunneling daemon. <https://github.com/OpenVPN/openvpn>. Online. 2020.
- [60] OpenVPN Reference Manual v. 2.4. <https://openvpn.net/community-resources/reference-manual-for-openvpn-2-4/>. Online. 2020.
- [61] T. Ylonen. The Secure Shell (SSH) Protocol Architecture. Technical report, January 2006.
- [62] Netfilter. <https://www.netfilter.org/>. Online. 2020.
- [63] Grafana: The tool for beautiful monitoring and metric analytics and dashboards. <https://github.com/grafana/grafana>. Online. 2020.
- [64] Joon-Myung Kang, Hadi Bannazadeh, and Alberto Leon-Garcia. Savi testbed: Control and management of converged virtual ict resources. In *2013 IFIP/IEEE International Symposium on Integrated Network Management (IM 2013)*, pages 664–667. IEEE, 2013.
- [65] Yu Emma Wang, Gu-Yeon Wei, and David Brooks. Benchmarking tpu, gpu, and cpu platforms for deep learning. *arXiv preprint arXiv:1907.10701*, 2019.
- [66] Yu Wang, Gu-Yeon Wei, and David Brooks. A systematic methodology for analysis of deep learning hardware and software platforms. *Proceedings of Machine Learning and Systems*, 2020:30–43, 2020.
- [67] NVIDIA Tesla V100 GPU Architecture Whitepaper. <https://images.nvidia.com/content/volta-architecture/pdf/volta-architecture-whitepaper.pdf>. Online. 2020.

- [68] K Villalobos, V J Ramírez, B Diez, J M Blanco, A Goñi, and A Illarramendi. A hierarchical storage system for industrial Time-Series data. In *2019 IEEE 17th International Conference on Industrial Informatics (INDIN)*, volume 1, pages 699–705. ieeexplore.ieee.org, July 2019.
- [69] M. Satyanarayanan, R. Schuster, M. Ebling, G. Fettweis, H. Flinck, K. Joshi, and K. Sabnani. An open ecosystem for mobile-cloud convergence. *IEEE Communications Magazine*, 53:63–70, 2015.